
20 MILLISECONDES

une image de Commodore 64, cycle par cycle

le VIC-II expliqué pas à pas, sans rien savoir d'avance

Treize instructions d'assembleur, dix expériences à lancer,
et toutes les captures d'écran prises sur une machine réelle :
de la bordure rouge au FLI.

Juillet 2026

Volume compagnon de *Au cœur du métal* — Commodore 64 & Ultimate 64

Rédaction assistée par IA (Claude) · audits croisés Codex et Claude Opus 5
sous la direction de **Johnny Piette**

Table des matières

Avant de commencer	4
Chapitre 0 — La machine vue de haut	5
La promesse	5
La visite de l'atelier	5
L'idée qui commande tout le livre	5
Une adresse peut être un levier	6
Mission 0 — installer l'atelier, lancer la bande-annonce	6
Au prochain chapitre	8
Chapitre 1 — Kit de survie 6502	9
La question	9
Les quatre premières	9
L'expérience : bordure rouge	9
L'amorce, expliquée une fois pour toutes	11
Le plus vite possible	11
Au prochain chapitre	12
Chapitre 2 — La trame : voir le temps	13
La question	13
Ce que fait le faisceau, en vingt millisecondes	13
Le casier qui dit l'heure	13
Guetter, plutôt qu'être prévenu	13
L'expérience	14
Le détail qui annonce le chapitre suivant	15
Au prochain chapitre	15
Chapitre 3 — La ligne : 63 battements	16
La question	16
L'unité de mesure de la machine	16
Ce que coûte chaque instruction	16
Une couleur prévue pour chaque ligne	16
L'expérience	17
La géométrie des déchirures	18
Au prochain chapitre	19
Chapitre 4 — La Bad Line : quand l'artiste réquisitionne le couloir	20
La question	20
Deux instructions pour fabriquer un chronomètre	20
L'expérience	20
Ce qui s'est passé dans le couloir	21
La contre-épreuve	21
La patience, et une prédiction vérifiée au pixel	22
Au prochain chapitre	24
Chapitre 5 — Les cellules : d'où viennent les caractères	25
La question	25
L'expérience — mille casiers	25
L'élégance du vieux routier : 1000 casiers en quatre passes de 256	27
Les codes écran ne sont PAS les codes ASCII	27
La Color RAM : mille casiers de plus, et un piège	27
Ces mille octets, c'est le butin de la Bad Line	28
Deuxième expérience — l'écran n'est qu'une adresse	28
Où poser la deuxième matrice ? (une décision, pas une recette)	28
Une lettre par cellule, à des places choisies	28
L'explication — $132 = 128 + 4$	30

Au prochain chapitre	31
Chapitre 6 — Les sprites : huit objets libres	32
La question	32
Le dessin : vingt-et-une lignes de 0 et de 1	32
L'expérience — un sprite, immobile, au centre	32
L'explication — pourquoi 64 ?	35
L'expérience — les huit, en rang	35
L'expérience — les huit voleurs	38
L'explication — ce que coûte vraiment un sprite	41
Au prochain chapitre	42
Chapitre 7 — Les compteurs cachés, et l'écran qui tombe	43
La question	43
Le signet du VIC	43
Le piège du décalage, et pourquoi il faut viser deux lignes en avance	43
L'expérience	44
Ce que vous venez d'acquérir	45
Au prochain chapitre	45
Chapitre 8 — Ouvrir la bordure	46
La question	46
La bordure n'est pas une zone : c'est un robinet	46
Deux valeurs, un seul interrupteur	46
Changer un interrupteur sans toucher aux autres	47
L'expérience	47
L'octet fantôme	49
Au prochain chapitre	50
Chapitre 9 — Le grand final : toutes les couleurs à la fois	51
La question	51
Ce qu'est vraiment le mode graphique	51
L'idée : une Bad Line par ligne	51
Trois tentatives, et ce que chacune enseigne	52
Où le VIC va chercher tout ça	53
L'expérience	53
La cicatrice	55
Pourquoi 24 rangées, et non 25	55
Ce que vous savez faire	56
Au prochain chapitre	56
Chapitre 10 — La même machine en 2026	57
La question	57
Une machine décrite, pas imitée	57
Le turbo : soixante-quatre fois plus vite, et toujours vingt millisecondes	57
Le monte-charge : la REU	58
La passerelle	58
La boucle est bouclée	59
Annexe — quatre pages à garder sous la main	60
Les seize couleurs	60
Les huit interrupteurs de \$d011	60
Les 63 cycles d'une ligne	60
Les notations de l'assembleur	61
Les treize instructions, et ce qu'elles coûtent	61
Les codes écran utilisés dans ce livre	62
Ce que tous les programmes supposent	62
Pour sortir d'un programme	62
Sources	63
Le volume compagnon	63
Le VIC-II, et le temps	63

Le processeur 63

Les puces que ce livre a laissées tranquilles 64

La machine de 2026 64

Le manuel d'origine 64

Avant de commencer

Ce livre attend de vous une seule chose : que vous ayez déjà programmé un peu. N'importe quoi, dans n'importe quel langage — si les mots *boucle* et *variable* ne vous font pas peur, vous avez tout ce qu'il faut. Il n'exige **ni** de connaître le Commodore 64, **ni** de connaître l'assembleur. Les deux s'apprendront ici, en chemin, et uniquement dans la mesure du nécessaire.

Le contrat est le même à chaque chapitre : une **question** qu'on peut voir à l'écran, une **expérience** — un programme court, complet, que vous tapez et lancez —, ce qu'on **observe**, puis l'**explication**. Jamais l'inverse. La théorie n'arrive qu'après que vos yeux ont constaté le phénomène.

Une dernière chose, pour la route : le livre se termine par une **annexe de quatre pages** qui rassemble ce qu'on cherche sans arrêt quand on programme — les seize couleurs, les huit interrupteurs du registre le plus important, la carte des 63 cycles d'une ligne, et les notations de l'assembleur. Rien ne vous oblige à la lire d'avance ; sachez seulement qu'elle est là.

Et une promesse : **toutes les images d'écran de ce livre sont vraies**. Chacune a été produite par le listing imprimé juste au-dessus d'elle, exécuté sur une machine réelle, et capturée telle quelle. Aucune n'a été dessinée, simulée ou retouchée.

Chapitre 0 — La machine vue de haut

La promesse

Toutes les vingt millisecondes, votre Commodore 64 dessine une image complète.

Pas « rafraîchit vaguement l'affichage » : il la dessine, entièrement, point par point, ligne par ligne — 312 lignes, du haut vers le bas, comme le faisceau d'électrons d'un téléviseur cathodique balayait jadis la dalle. Cinquante fois par seconde, sans jamais une exception, sans jamais un retard. Pendant que votre programme tourne, pendant qu'il plante, pendant que vous ne faites rien : l'image, elle, est livrée. Toutes les vingt millisecondes.

Ce livre raconte ce qui se passe pendant **une seule** de ces images. Vingt millisecondes, ralenties jusqu'à ce que chaque microseconde devienne visible, puis chaque **cycle** — le battement élémentaire de la machine, un peu plus d'un millionième de seconde. Une image, c'est exactement 19 656 battements. À la fin du livre, vous saurez lire ces battements : combien en coûte chaque instruction, à qui ils appartiennent, lesquels vous seront volés et quand — et surtout, vous saurez vous glisser dans les interstices pour faire faire à la machine des choses que ses concepteurs n'avaient pas prévues.

C'est un voyage qui se mérite mais qui paie comptant : dès ce chapitre, vous lancerez un programme. Au chapitre suivant, vous en écrirez un.

La visite de l'atelier

Ouvrez la machine — mentalement, ça suffira. À l'intérieur, un atelier. Petit, dense, et organisé autour d'une contrainte unique dont tout le reste découle.

La réserve : 64 kilo-octets de mémoire. Imaginez 65 536 casiers alignés, numérotés de 0 à 65535, contenant chacun un octet — un nombre entre 0 et 255. C'est tout. Le texte de votre programme, les images, les sons, vos variables : tout ce qui existe dans cette machine vit dans ces casiers.

Le couloir unique : le bus. Pour lire ou écrire un casier, il faut emprunter le couloir qui mène à la réserve. Et ce couloir n'a qu'une seule voie : *un* accès à la fois, environ un par microseconde. Retenez ce couloir. La moitié de ce livre — et les plus beaux trucages du Commodore 64 — sont des histoires de couloir.

Les deux artisans. Dans l'atelier travaillent deux ouvriers, et ils n'ont pas le même rapport au temps :

- Le **6510**, le processeur. C'est lui qui exécute vos instructions, une par une, consciencieusement. Il calcule, compare, déplace des octets. Il a une qualité rare : il est *patient*. Si on le fait attendre, il attend.
- Le **VIC-II**, la puce vidéo. L'artiste de l'atelier — et un artiste sous contrat impitoyable : il doit livrer une image toutes les vingt millisecondes. Pas « dès que possible » : *à l'heure*. Le faisceau qui balaie l'écran n'attend personne ; si le VIC manquait son rendez-vous, l'image se déchirerait à l'instant même. Lui n'a pas le droit d'être patient.

Les seconds rôles. Deux autres puces méritent un salut : le **SID**, le musicien (trois voix, un caractère légendaire), et les deux **CIA**, les portiers — clavier, joysticks, horloges. Ce livre les laissera tranquilles : notre affaire, c'est l'image.

L'idée qui commande tout le livre

Deux artisans, un seul couloir : il faut bien partager. Et le partage n'est pas négocié à l'amiable — il découle de la contrainte : *le VIC ne peut pas être en retard, le processeur peut attendre*.

Alors la machine coupe chaque microseconde en deux moitiés : une pour le VIC, une pour le 6510. Chacun sa demi-microseconde, chacun ses accès à la réserve, et dans le cas normal personne ne marche sur personne. Vous découvrirez au chapitre 4 ce qui arrive quand la moitié du VIC ne lui suffit plus — c'est l'un des secrets les mieux enfouis de la machine, et la source de ses plus célèbres bizarreries.

En attendant, retenez la phrase qui résume ce rapport de force, parce que tout le livre en découle :

Dans un Commodore 64, le temps appartient à la puce vidéo. Le processeur travaille dans le temps que le VIC lui laisse.

Une adresse peut être un levier

Un dernier secret d'atelier avant de passer aux travaux pratiques. Parmi les 65 536 casiers, quelques dizaines **ne sont pas de la mémoire**. Ils sont câblés directement sur les puces : écrire dedans, c'est *tirer un levier*. Le casier 53280 (les habitués écrivent son numéro en hexadécimal : \$d020) est câblé sur le VIC — écrivez-y un nombre de 0 à 15, et la bordure de l'écran change de couleur, immédiatement. Pas besoin de comprendre l'assembleur pour trouver ça remarquable : *changer une couleur, c'est écrire dans une case mémoire*.

Toute la programmation de ce livre tient dans cette idée : savoir **quoi écrire, dans quel casier — et à quel moment**. Les deux premiers points, c'est le chapitre 1. Le troisième, c'est tout le reste du livre.

Mission 0 — installer l'atelier, lancer la bande-annonce

Il vous faut deux outils, gratuits et disponibles sur tous les systèmes :

- **VICE**, l'émulateur (programme x64sc, l'émulation la plus fidèle) — vice-emu.sourceforge.io. Si vous possédez une machine réelle — un C64 d'époque, ou l'une de ses rééditions matérielles (Ultimate 64, C64 Ultimate) — encore mieux : tout ce livre a été testé sur le vrai métal.
- **ACME**, l'assembleur — le traducteur qui transforme un texte source en programme exécutable. Une seule commande à connaître : `acme fichier.a`. Prenez la **version 0.95 ou plus récente** (ce livre a été assemblé avec la 0.97) : les versions antérieures ne connaissent pas la forme de boucle utilisée aux chapitres 7 et 9.

Tous les programmes de ce livre sont fournis à côté de lui, un dossier par chapitre (`ch00/`, `ch01/...`), avec la capture d'écran que chacun a réellement produite. Vous pouvez donc les taper — c'est formateur — ou partir du fichier. Voici la bande-annonce du livre :

```

;-----
; 20 MILLISECONDES – chapitre 0 : la bande-annonce
;
; Vous ne comprendrez pas encore ce listing. C'est normal.
; C'est la destination : à la fin du livre, il n'aura plus
; aucun secret pour vous – et vous saurez faire bien mieux.
;
; Assembler :   acme teaser.a
; Lancer :      LOAD"TEASER",8,1 puis RUN
;-----
!to "teaser.prg", cbm

* = $0801                ; amorce BASIC : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence : plus personne ne nous interrompt

boucle ldx $d012        ; à quelle LIGNE en est le faisceau ?
      lda couleurs,x    ; la couleur prévue pour cette ligne...
      sta $d020        ; ...peinte sur la bordure
      sta $d021        ; ...et sur le fond
      jmp boucle        ; pour toujours

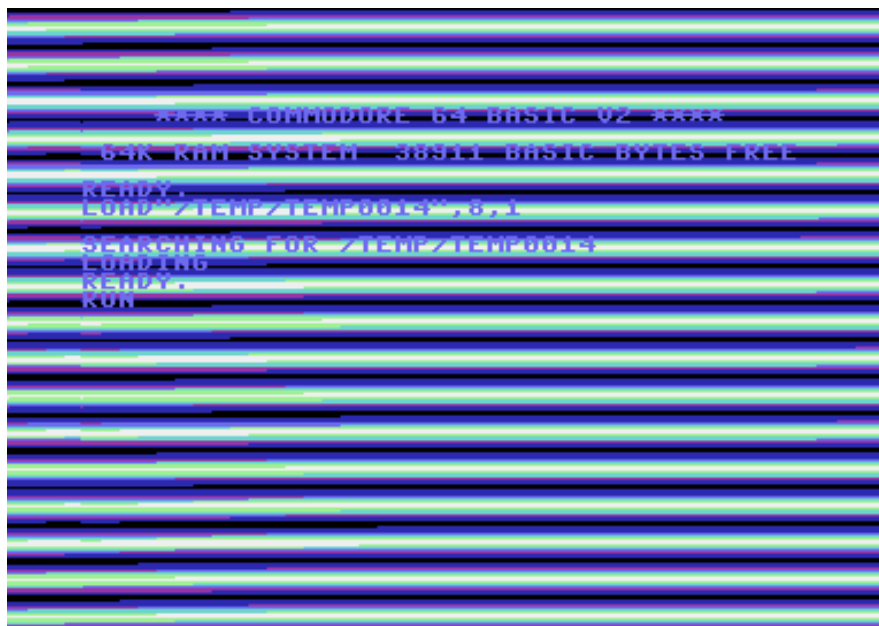
; une vague de 16 couleurs, sombre -> claire -> sombre
!macro vague { !byte 0,6,6,4,14,3,13,1,1,13,3,14,4,6,6,0 }

couleurs                ; 256 lignes = 16 vagues
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague
    +vague

```

Assemblez (acme teaser.a), lancez le teaser.prg obtenu (glissez-le sur la fenêtre de VICE, ou LOAD"TEASER",8,1 puis RUN sur une vraie machine).

Ce qu'on observe :



Capture réelle (C64 Ultimate, sortie vidéo native). Le texte de démarrage du BASIC est toujours là — des vagues de

couleur le traversent, du haut en bas de l'écran, bordure comprise.

Regardez bien : la couleur change à **chaque ligne de balayage**. Pas à chaque caractère, pas à chaque « case » — à chaque ligne du faisceau, 312 fois par image, 50 fois par seconde. Aucun langage « normal » ne sait faire ça : le BASIC du C64 exécute à peine quelques dizaines d'instructions pendant qu'une image *entière* est dessinée. Ici, le programme discute avec le faisceau *pendant* qu'il balaie — et il ne fait que six instructions.

Vous ne savez pas encore lire $\$d012$, ni pourquoi la table couleurs fait 256 entrées, ni ce que sei interdit exactement. C'est toute la promesse : **chapitre 2** pour le faisceau et $\$d012$, **chapitre 3** pour comprendre — à la microseconde près — pourquoi les vagues « penchent » légèrement sur les bords de l'image que vous avez sous les yeux. Rien dans cette capture n'est un défaut : tout y est déjà une leçon de timing.

Sous le capot — pour les lecteurs pressés de vérifier : le partage du couloir en demi-microsecondes (signaux $\phi 0/\phi 2$, AEC et BA) est spécifié dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §1 — le volume de référence dont ce livre est le guide de visite : une synthèse des sources d'origine, austère et faite pour être consultée, pas apprise. Chaque chapitre se terminera par un renvoi de ce genre : ici, les phénomènes ; là-bas, les règles exactes et leurs sources.

Au prochain chapitre

Vous avez lancé le programme d'un autre. Au chapitre 1, vous écrirez le vôtre : treize instructions d'assembleur — pas une de plus dans tout ce livre — et le premier levier tiré de vos mains.

Chapitre 1 — Kit de survie 6502

La question

Vous avez lancé le programme d'un autre. Comment écrit-on le sien ?

Il va falloir parler à la machine dans sa langue — et c'est ici que les livres perdent d'ordinaire la moitié de leurs lecteurs, dans un chapitre de trente pages sur les modes d'adressage. Nous allons faire autrement. Ce livre entier n'utilise que **treize instructions**. Pas treize pour commencer : treize en tout, jusqu'à la dernière page, FLI compris. Vous en apprendrez **quatre** dans ce chapitre, et vous aurez déjà de quoi écrire un programme qui marche.

L'idée à laquelle il faut renoncer, d'abord : l'assembleur n'est pas un langage à apprendre, avec sa grammaire et ses idiomes. C'est un **traducteur**, et un traducteur sans imagination : chaque ligne que vous écrivez devient exactement un ordre pour le processeur, ni plus, ni moins. `lda #2` devient deux octets. Il n'y a rien en dessous. C'est précisément ce dépouillement qui va vous donner le contrôle du temps, au chapitre 3.

Les quatre premières

Le processeur possède trois cases à lui, minuscules — un octet chacune — dans lesquelles il tient ce sur quoi il travaille en ce moment. On les appelle des **registres** : A, X et Y. A est le registre de travail, l'accumulateur ; X et Y attendront les chapitres 3 et 5. Tout ce que fait le 6502 consiste à charger un octet dans un registre, à le triturer, puis à le déposer quelque part.

Nouvelle instruction — LDA (« LoaD A », charger A) : met un octet dans le registre A. `lda #2` charge la valeur 2. `lda $d020` charge le contenu du casier `$d020`. Le # change tout : avec lui une valeur, sans lui une adresse. C'est la seule confusion vraiment coûteuse de l'assembleur 6502 — si un programme se comporte bizarrement, c'est le premier endroit où regarder.

Nouvelle instruction — STA (« STore A », ranger A) : dépose le contenu de A dans un casier. `sta $d020` écrit dans le casier 53280. Il n'existe pas de `sta #2` : on ne range rien « dans une valeur », l'idée n'a pas de sens.

Deux mots sur cette écriture `$d020`. Le \$ annonce un nombre en **hexadécimal**, c'est-à-dire compté par paquets de seize au lieu de dix. Ce n'est pas une coquetterie d'initiés : la machine est bâtie sur des paquets de bits, et en hexadécimal les frontières tombent rond. `$d020`, `$d021`, `$d022`... se suivent visiblement, alors que 53280, 53281, 53282 n'évoquent rien. Vous n'avez rien à convertir mentalement : lisez `$d020` comme un nom propre, celui du casier de la bordure.

Nouvelle instruction — JMP (« JuMP », sauter) : continue l'exécution ailleurs. `jmp boucle` reprend à l'endroit que vous avez baptisé `boucle`. C'est le goto de la machine — et ici, personne ne vous le reprochera : c'est notre seul outil de répétition jusqu'au chapitre suivant, qui en ajoutera un plus fin.

Nouvelle instruction — RTS (« ReTurn from Subroutine », revenir) : rend la main à qui vous avait appelé. Comme c'est le BASIC qui a lancé notre programme (avec son SYS), `rts` nous ramène au READY.

L'expérience : bordure rouge

Voici le programme entier. Ne vous laissez pas intimider par la douzaine d'octets du début : je les explique juste après, une fois pour tout le livre.

```

;-----
; 20 MILLISECONDES – chapitre 1 : votre premier programme
;
; Trois instructions. La bordure devient rouge, et le BASIC
; reprend la main comme si rien ne s'était passé.
;
; Assembler :  acme bordure.a
; Lancer :      LOAD"BORDURE",8,1  puis  RUN
;-----
!to "bordure.prg", cbm

; --- l'amorce : une ligne de BASIC, rangée à la main dans des casiers ---
* = $0801
!byte $0b,$08          ; où commence la ligne SUIVANTE ($080b)
!byte $0a,$00          ; le numéro de la ligne : 10
!byte $9e              ; le mot SYS, rangé en un seul octet
!byte $32,$30,$36,$34  ; les caractères "2 0 6 4"
!byte $00              ; fin de la ligne
!byte $00,$00          ; fin du programme BASIC

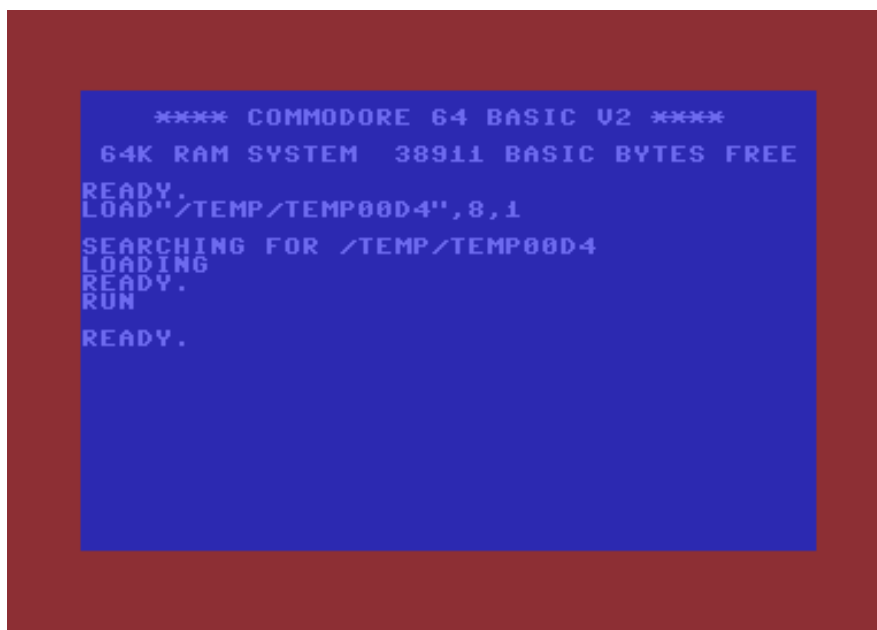
; --- notre programme, à l'adresse 2064 ($0810) ---
* = $0810
    lda #2              ; A <- la VALEUR 2 (rouge)
    sta $d020           ; ...déposée dans le casier-levier de la bordure
    rts                ; et on rend la main au BASIC

```

Deux lignes de ce listing ne sont pas des instructions mais des ordres donnés à l'assembleur : `* = $0801` et `* = $0810` signifient « range la suite à partir de cette adresse » — l'étoile désigne l'adresse courante, ce n'est pas une multiplication. Toutes les notations de ce genre sont rassemblées dans l'annexe.

Trois lignes utiles. Assemblez (`acme bordure.a`), lancez le `bordure.prg` obtenu.

Ce qu'on observe :



Capture réelle. Le chemin bizarre dans la ligne de chargement est celui de notre machine d'essai, qui reçoit les programmes par le réseau plutôt que par disquette — chez vous, ce sera le nom de votre fichier.

Deux choses méritent votre attention. D'abord, le `READY.` : notre programme est **fini**, le BASIC a repris la main, vous pouvez taper `PRINT 2+2` comme si rien ne s'était passé. Ensuite — et c'est la leçon du chapitre — **la bordure est restée rouge**. Personne ne la maintient rouge. Aucune boucle ne la repeint cinquante fois par seconde. Nous avons déposé un 2 dans un casier, et ce 2 y demeure : le VIC le relit tout seul, pour chaque pixel de bordure de chaque image, jusqu'à ce que quelqu'un écrive autre chose.

(Les seize couleurs du Commodore 64 et leurs numéros sont rassemblés dans l'annexe, à la fin du livre : c'est la page que vous consulterez le plus souvent.)

C'est la nature même d'un **casier-levier** : on ne lui donne pas des ordres, on le *positionne*. Retenez-le, parce qu'à partir du chapitre 7 nous ferons exactement le contraire — réécrire un registre encore et encore, à des instants choisis, pour tromper le VIC. Mais pour ça, il faudra savoir *quand*.

L'amorce, expliquée une fois pour toutes

Cette douzaine d'octets au début vous suivra dans tous les programmes du livre. Elle est là parce qu'on ne peut pas lancer directement du code machine depuis l'écran d'accueil du C64 : il faut passer par le BASIC. Alors on lui fabrique un programme minuscule — 10 SYS 2064 — non pas en le tapant, mais en rangeant à la main les octets qui le représentent.

Car un programme BASIC, dans cette machine, n'est rien d'autre que des octets dans des casiers, exactement comme le reste. La ligne 10 SYS 2064 s'écrit : l'adresse de la ligne suivante (deux octets), le numéro de ligne 10 (deux octets), un octet \$9e qui *est* le mot SYS (le BASIC range ses mots-clés en un seul octet chacun — c'est pour ça qu'un programme BASIC tient dans si peu de place), puis les quatre caractères « 2064 », puis des zéros pour dire « c'est fini ». Le SYS du BASIC saute à l'adresse 2064, où nos trois instructions attendent. 2064, c'est \$0810 : juste après l'amorce.

Vous n'aurez plus jamais à y penser. Copiez ces douze octets en tête de chaque programme, et considérez-les comme le « bonjour » d'usage adressé au BASIC.

Le plus vite possible

Une dernière expérience, et elle va poser la question de tout le livre. Que se passe-t-il si on change la couleur de la bordure **aussi vite que la machine en est capable** ?

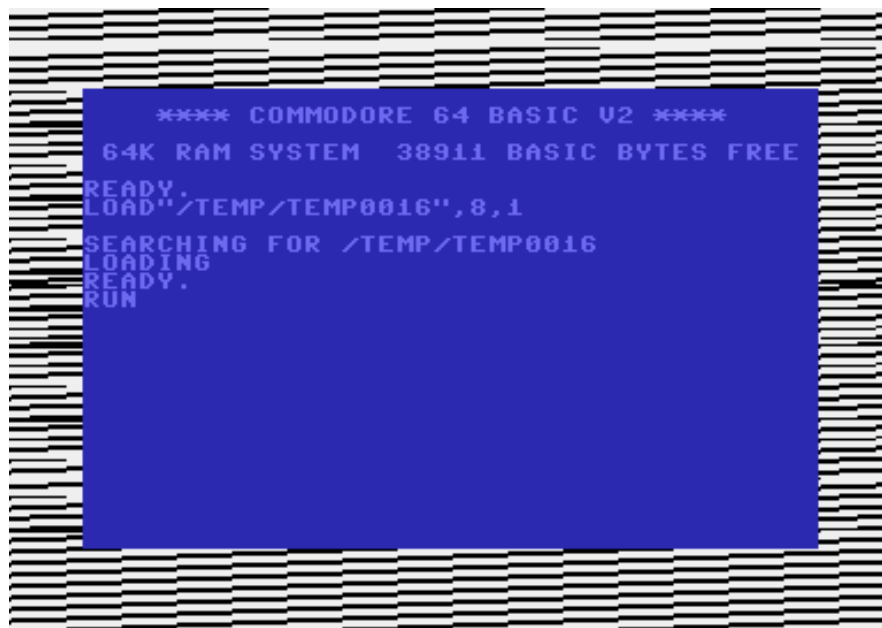
```
!to "stroboscope.prg", cbm

* = $0801                ; l'amorce du chapitre : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
boucle lda #0            ; noir
      sta $d020
      lda #1             ; blanc
      sta $d020
      jmp boucle         ; et on recommence, sans jamais s'arrêter
```

Votre intuition dit : la bordure va clignoter — sans doute trop vite pour l'œil, donc elle paraîtra grise. Lancez.

Ce qu'on observe :



Capture réelle. La bordure n'est ni noire, ni blanche, ni grise : elle est rayée.

Des stries. Pas un clignotement, pas un gris uniforme : des bandes noires et blanches parfaitement dessinées, immobiles. Regardez-les bien, elles contiennent tout ce que ce livre a à dire.

Ce que vous voyez, c'est une **course** entre deux choses qui vont à peu près à la même vitesse. Notre boucle est si courte qu'elle repeint la bordure **huit fois pendant qu'une seule ligne d'écran se dessine** — noir,

blanc, noir, blanc, quatre fois de suite. Chaque ligne est donc découpée en huit tranches alternées, et comme la boucle et le balayage ne tombent jamais parfaitement en phase, le découpage se décale un peu à chaque ligne : d'où ces rayures. (Vous saurez recompter ce « huit » vous-même au chapitre 3.)

Vous venez de rencontrer le personnage principal du livre : **le faisceau**. Il n'attend pas votre programme, il n'accélère pas pour lui, il avance à sa vitesse — et tout l'artisanat du Commodore 64 consiste à écrire au bon endroit *pendant qu'il passe*.

Nous savons désormais écrire dans un casier. Il nous manque exactement une chose : savoir **où en est le faisceau**.

Sous le capot — les 47 casiers-leviers du VIC, leur adresse et le rôle de chacun de leurs bits, sont donnés en une seule table dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §2 ; la palette des seize couleurs et la façon dont elles sont fabriquées (par mélange de signaux, sans aucun rouge-vert-bleu) au §3.

Au prochain chapitre

Le C64 possède un casier qui répond à cette question, en permanence, gratuitement. Nous allons le lire — et nos rayures aléatoires vont devenir une bande de couleur parfaitement immobile, posée exactement là où nous l'aurons décidé.

Chapitre 2 — La trame : voir le temps

La question

Nos rayures du chapitre 1 étaient jolies mais subies. Nous voulons décider : une bande de couleur, ici, de cette hauteur, immobile. Pour ça, il faut savoir où en est le faisceau.

Ce que fait le faisceau, en vingt millisecondes

Voici le trajet complet, et il n'y a rien de plus à savoir pour ce chapitre. Le faisceau part en haut à gauche, traverse l'écran vers la droite, revient à gauche une ligne plus bas, et recommence. En PAL — les machines européennes — il dessine ainsi **312 lignes**, puis remonte tout en haut et repart pour l'image suivante. Cinquante fois par seconde.

Toutes les lignes ne sont pas visibles : les premières et les dernières tombent hors de la dalle, héritage des téléviseurs qui avaient besoin de ce temps mort pour ramener leur faisceau en haut. Sur les lignes qui restent, une bande de bordure à gauche, une à droite, et au milieu la fenêtre d'affichage : 320 pixels de large, 200 de haut.

Ce qui nous intéresse, c'est que ce trajet est **parfaitement régulier**. Le faisceau ne saute pas, n'accélère pas, ne rate jamais son rendez-vous. C'est l'horloge la plus fiable de la machine — et le C64 nous laisse la consulter.

Le casier qui dit l'heure

Le casier \$d012 — le compteur de lignes. À tout instant, il contient le numéro de la ligne que le faisceau est en train de dessiner : une valeur qui change toute seule, **environ 15 600 fois par seconde** (312 lignes, 50 images). On peut aussi y écrire — cela sert à programmer une interruption, que ce livre n'utilise pas ; nous nous contenterons donc de le **lire**.

Une honnêteté tout de suite : un casier ne contient qu'un octet, donc au maximum 255, et il y a 312 lignes. Le numéro complet a besoin d'un neuvième bit, qui vit ailleurs (dans \$d011). Passé la ligne 255, \$d012 repart donc de zéro pour la fin de l'image. Nous resterons sagement en dessous de 255 dans ce chapitre — c'est-à-dire dans presque tout l'écran visible.

Guetter, plutôt qu'être prévenu

Notre stratégie sera la plus simple du monde : lire \$d012 en boucle jusqu'à ce qu'il affiche la valeur attendue. On appelle ça **guetter** (les Anglo-Saxons disent *polling*). Le programme ne fait rien d'autre que regarder passer le faisceau, et agit dès qu'il arrive.

Ce n'est pas la méthode des professionnels : le C64 sait *prévenir* le processeur quand une ligne donnée est atteinte, ce qui laisse tout le temps intermédiaire libre pour faire autre chose. Mais ce mécanisme demande d'installer un gestionnaire d'interruption, de sauver des registres, de comprendre des détours qui n'ont rien à voir avec le VIC. Ce livre a fait un choix radical : **tout, jusqu'au FLI du chapitre 9, se fera en guettant**. Vous verrez que c'est amplement suffisant — et que ça garde le programme lisible d'un seul coup d'œil.

Il faut cependant faire taire quelqu'un. Soixante fois par seconde, le système d'exploitation du C64 interrompt ce qui tourne pour scruter le clavier, faire clignoter le curseur, avancer son horloge interne. Une soixantaine de microsecondes volées, au pire moment : notre guet raterait sa ligne.

Nouvelle instruction — SEI (« **SEt Interrupt disable** ») : ferme la porte. Après sei, plus personne n'interrompt notre programme. Le clavier ne répond plus (RUN/STOP compris) — pour reprendre la main, il faudra RUN/STOP + RESTORE, qui passe par une autre porte, ou éteindre. C'est le prix du silence, et tous les programmes de ce livre le paient.

Nouvelle instruction — CMP (« CoMPare », comparer) : compare le registre A à une valeur, sans rien modifier. `cmp #$80` demande : « A vaut-il 128 ? »

Nouvelle instruction — BNE (« Branch if Not Equal », sauter si différent) : saute si la comparaison précédente **n'était pas** une égalité. C'est notre « tant que » : `cmp #$80` suivi de `bne haut` signifie « tant que ce n'est pas 128, retourne guetter ».

L'expérience

```
!to "bande.prg", cbm

* = $0801                ; 10 SYS 2064 (l'amorce du chapitre 1)
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; le systeme ne nous interrompra plus

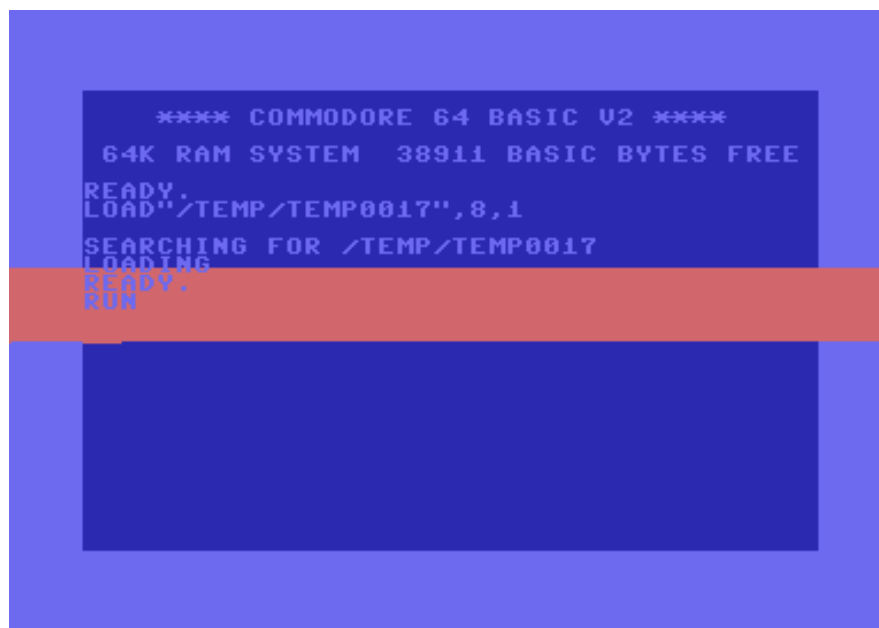
; --- guetter la ligne $80 (128), au milieu de l'ecran ---
haut    lda $d012        ; ou en est le faisceau, LA, maintenant ?
        cmp #$80        ; est-il arrive a la ligne 128 ?
        bne haut        ; non : on regarde encore
        lda #$0a        ; oui : rouge clair
        sta $d020        ;   bordure
        sta $d021        ;   et fond

; --- guetter la ligne $a0 (160), 32 lignes plus bas ---
bas     lda $d012
        cmp #$a0
        bne bas
        lda #$0e        ; bleu clair : la bordure d'origine
        sta $d020
        lda #$06        ; bleu : le fond d'origine
        sta $d021

        jmp haut        ; et on recommence, trame apres trame
```

Quinze instructions en tout, dont aucune n'est nouvelle sauf les trois du chapitre. Notez aussi le motif à retenir, qu'on retrouvera partout dans ce livre : **charger \$d012, comparer, boucler si ce n'est pas encore l'heure.**

Ce qu'on observe :



Capture réelle. La bande traverse la bordure aussi bien que la fenêtre d'affichage : nous peignons le décor, pas les caractères — le texte du BASIC continue de vivre sa vie par dessus.

Une bande. Immobile. Trente-deux lignes de haut, posée où nous l'avons demandé. Nous n'avons pas dessiné un rectangle : nous avons changé la couleur du décor **pendant deux courtes fenêtres de temps**, et

le faisceau, en passant, a fait le reste. Le rectangle n'existe nulle part en mémoire.

Prenez un instant pour mesurer ce qui vient de se produire. Le C64 n'a aucune fonction pour tracer une bande de couleur en travers de la bordure — c'est même précisément ce que son matériel *ne sait pas faire*. Nous l'avons obtenu en écrivant deux fois dans un casier, au bon moment. C'est ça, la programmation du C64 : la machine ne connaît pas les figures, elle connaît les instants.

Le détail qui annonce le chapitre suivant

Approchez-vous de la capture, au bord gauche de la bande, à l'endroit où le rouge commence. La transition n'est pas parfaitement franche : la ligne du haut n'a pas basculé exactement au même endroit que les autres. Ce n'est ni un défaut de la capture, ni un défaut de votre machine. Quand notre `lda $d012` lit enfin 128, le faisceau **est déjà en train de dessiner** la ligne 128 : il a avancé pendant que nous comparions, pendant que nous chargions la couleur, pendant que nous l'écrivions. La bande commence donc quelques dizaines de pixels après le bord gauche — là où le faisceau se trouvait quand notre `sta` a eu lieu.

Autrement dit : nous savons désormais viser une ligne. Nous ne savons pas encore viser un **endroit dans** la ligne. Or c'est là que se cachent tous les trucages du Commodore 64.

Sous le capot — le nombre de lignes et de cycles pour chaque variante de la puce (PAL, NTSC, et leurs révisions), les dimensions exactes de la fenêtre d'affichage, et le neuvième bit du compteur de lignes sont dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §4 ; le mécanisme d'interruption qui remplace notre `guet`, au §14.

Au prochain chapitre

Nous allons descendre d'un cran dans le zoom : à l'intérieur d'une seule ligne. Vous découvrirez qu'elle dure exactement 63 battements, que chacune de nos instructions en consomme un nombre connu d'avance — et vous saurez enfin lire, ligne par ligne, la bande-annonce du chapitre 0.

Chapitre 3 — La ligne : 63 battements

La question

Au chapitre précédent, notre bande a raté son bord gauche de quelques dizaines de pixels. Nous avons dit « le faisceau avance pendant qu'on travaille ». Combien, exactement ?

C'est la question la plus rentable de tout le livre. Y répondre demande de descendre d'un cran dans le zoom : jusqu'ici nous comptons les lignes, nous allons maintenant compter à l'intérieur d'une ligne.

L'unité de mesure de la machine

Une ligne d'écran, sur un C64 européen, dure exactement **63 cycles**.

Le cycle, c'est le battement du processeur : 1,015 millionième de seconde très exactement — appelons ça une microseconde et n'y revenons plus. C'est aussi, et c'est là que tout se joue, la durée pendant laquelle le faisceau parcourt exactement **huit pixels**. Un cycle, huit pixels : cette équivalence est la règle de trois la plus utile du Commodore 64. Elle dit que le temps et l'espace sont la même chose vus sous deux angles — un retard de dix cycles se lit à l'écran comme un décalage de quatre-vingts pixels.

Faisons le compte de notre image : 63 cycles par ligne, 312 lignes, cela fait 19 656 cycles par image. Vous reconnaissez le nombre : ce sont les 19 656 battements annoncés au chapitre 0. À raison d'un peu plus d'une microseconde chacun, ils font nos vingt millisecondes ($19\,656 \times 1,015\ \mu\text{s} = 19,95\ \text{ms}$: voilà d'où vient le titre de ce livre). La boucle est bouclée : nous avons enfin les deux bouts de l'échelle, du battement à l'image.

Ce que coûte chaque instruction

Voici le tarif des instructions que nous connaissons. Ces nombres ne s'inventent pas et ne se devinent pas : ils sont câblés dans le processeur, publiés, et vérifiables au cycle près.

Instruction	Ce qu'elle fait	Coût
lda #2	charger une valeur	2 cycles
lda \$d012	charger le contenu d'un casier	4 cycles
lda couleurs,x	charger la case n° X d'une table	4 cycles
sta \$d020	ranger dans un casier	4 cycles
cmp #\$80	comparer à une valeur	2 cycles
bne boucle	sauter si différent	2 cycles si on ne saute pas, 3 si on saute
jmp boucle	sauter	3 cycles
sei	fermer la porte aux interruptions	2 cycles

Reprenons le guet du chapitre 2 avec cette grille en main : lda \$d012 (4), cmp #\$80 (2) et bne (3 quand il boucle) font **9 cycles par tour de guet**. Neuf cycles, c'est 72 pixels.

Voilà l'explication du bord raté : quand notre boucle constate enfin l'arrivée de la ligne 128, le faisceau peut déjà avoir avancé de neuf cycles dans cette ligne — et il continue d'avancer pendant les 2 cycles du lda #\$0a et les 4 du sta. Notre bande ne pouvait pas commencer au bord gauche. Elle commençait là où la machine en était.

Rien de tout ceci n'est un défaut. C'est la règle du jeu, et elle est *connue d'avance* : c'est exactement pour ça qu'on peut la contourner. Les chapitres 7 à 9 ne feront rien d'autre que compter des cycles pour tomber au bon endroit.

Une couleur prévue pour chaque ligne

Nous avons de quoi comprendre la bande-annonce du chapitre 0. Il manque un outil, et un seul : un deuxième registre pour transporter le numéro de ligne, et la capacité de s'en servir comme d'un index dans une table.

Nouveau registre et nouvelle instruction — LDX (« Load X ») : comme `lda`, mais pour le registre X. `ldx $d012` met le numéro de la ligne courante dans X.

Nouveau mode — l'indexé : `lda couleurs,x` : charge non pas le contenu de `couleurs`, mais celui de la case X de la table qui commence à `couleurs`. Si X vaut 128, on lit le 129^e octet de la table. Une instruction, quatre cycles, et vous avez le tableau indexé des langages ordinaires.

Ces deux-là forment le **motif signature** de ce livre :

```
ldx $d012      ; où en est le faisceau ?
lda couleurs,x ; qu'avais-je prévu pour cette ligne ?
```

Retenez-le : nous le retrouverons au chapitre 7 pour faire tomber l'écran, et au chapitre 9 pour le FLI. C'est la façon la plus économique qui existe de faire varier quelque chose ligne par ligne — parce que **tout le calcul a été fait d'avance**, par l'assembleur, à la fabrication du programme. Pendant l'affichage, il ne reste plus qu'à lire.

L'expérience

```
!to "degrade.prg", cbm

* = $0801      ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei      ; silence

boucle ldx $d012      ; 4 cycles : X <- la ligne en cours
      lda couleurs,x ; 4 cycles : la couleur prevue pour CETTE ligne
      sta $d020      ; 4 cycles : bordure
      sta $d021      ; 4 cycles : fond
      jmp boucle     ; 3 cycles : et on recommence
                        ; total : 19 cycles par tour de boucle

; --- la table : une couleur par ligne ---
; !align garantit qu'elle commence au debut d'une page de 256 octets :
; sans ca, « lda couleurs,x » couterait parfois 5 cycles au lieu de 4,
; et notre boucle ne battrait plus regulierement.
!align 255, 0

; une vague de 16 teintes : sombre -> claire -> sombre
!macro vague { !byte 0,6,6,4,14,3,13,1,1,13,3,14,4,6,6,0 }

couleurs      ; 16 vagues = 256 lignes
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
+vague
```

Notez au passage `sta $0400,x` : le mode indexé du chapitre 3 fonctionne aussi **en écriture**. « La case n° X de la table » sert à ranger comme à charger — c'est ce qui rend une boucle de remplissage aussi courte.

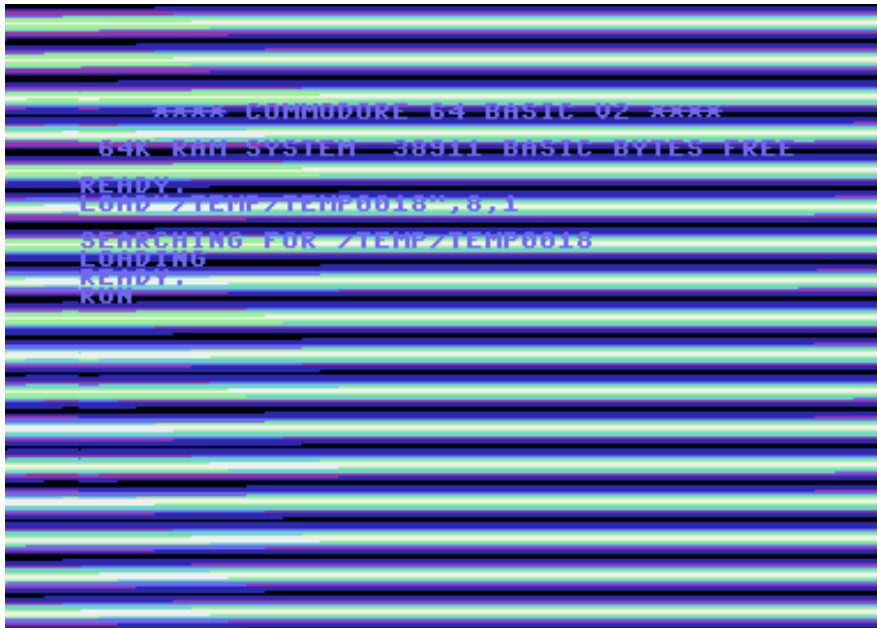
Deux mots sur les dernières lignes, qui ne sont pas des instructions. `!macro` et `+vague` sont des commodités de l'assembleur : on décrit une vague de seize teintes, on la réclame seize fois, l'assembleur écrit les 256 octets.

Et ces seize teintes ne sont pas au hasard : 0, 6, 6, 4, 14, 3, 13, 1 puis les mêmes en sens inverse. Ce sont des numéros de couleurs — noir, bleu, rose, bleu clair, cyan, vert clair, blanc — rangés **du plus sombre au**

plus clair, puis redescendus. Deux propriétés en découlent : la montée est douce, parce qu'on suit l'échelle de luminosité de la palette et non l'ordre des numéros ; et la vague se referme sur elle-même, parce qu'elle finit comme elle a commencé. Les seize répétitions se raccordent donc sans couture. Le processeur, lui, n'en saura rien : il trouvera juste une table toute faite. **Tout ce qui peut être calculé à la fabrication ne coûtera rien à l'affichage** — c'est le principe qui rendra le chapitre 9 possible.

Quant au `!align`, il mérite son commentaire dans le listing. Notre table fait exactement 256 octets ; si elle commence au début d'une page de 256, alors `lda couleurs, x` reste dans cette page quelle que soit la valeur de `x`. Sinon, pour les valeurs élevées de `x`, le processeur doit corriger l'adresse et l'instruction coûte **5 cycles au lieu de 4**. Une boucle qui bat parfois 19 cycles et parfois 20, c'est une boucle qui dérive. Nous venons de payer une ligne d'assembleur pour acheter de la régularité.

Ce qu'on observe :



Capture réelle. Comparez avec la bande-annonce du chapitre 0 : ce sont les mêmes six instructions, à une ligne près — celle du chapitre 0 n'avait pas de `!align`, et c'est précisément pourquoi ses vagues étaient un peu plus désordonnées que celles-ci.

La géométrie des déchirures

Reste à expliquer ce que vous voyez vraiment — parce que les vagues ne sont pas des bandes propres, elles ont des bords en escalier et des accidents.

Notre boucle dure 19 cycles. Une ligne en dure 63. Donc la boucle tourne **trois fois par ligne**, à peu près : trois fois 19 font 57, il reste 6 cycles de rabiot. « À peu près » est le mot important — 63 n'est pas un multiple de 19, et ce reste de 6 cycles se cumule d'une ligne à la suivante. Le moment où la boucle attrape le changement de ligne dérive donc, ligne après ligne : c'est ce glissement qui dessine l'escalier des bords.

Et les trois passages dans la même ligne ? Ils lisent tous les trois le même numéro de ligne, donc écrivent trois fois la même couleur : invisible. Mais celui qui chevauche le changement de ligne, lui, écrit l'ancienne couleur au début de la ligne nouvelle — d'où ces petits segments décalés, à droite, aux transitions.

Notez la conclusion, qui est le programme des six chapitres suivants : **notre boucle est un peu plus lente que ce qu'il faudrait, et surtout, elle n'est pas alignée sur la ligne**. Pour obtenir des bandes parfaitement nettes, il faudrait savoir *exactement* combien de cycles s'écoulent entre deux écritures, et se recalculer sur chaque début de ligne. Nous apprendrons les deux.

Sous le capot — le coût en cycles de chaque instruction du 6510, mode d'adressage par mode d'adressage et étape par étape, est dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 3 (d'après le document *64doc*) : c'est là que se vérifient les huit nombres du tableau ci-dessus, et le cycle supplémentaire des franchissements de page.

Au prochain chapitre

Mais il y a un obstacle, et il est de taille : le processeur n'est pas seul dans la machine. Une fois toutes les huit lignes, le VIC lui prend le couloir, et notre boucle si soigneusement chronométrée perd d'un coup une quarantaine de cycles. Ça a un nom, ça se voit à l'œil nu, et c'est le cœur du Commodore 64.

Chapitre 4 — La Bad Line : quand l'artiste réquisitionne le couloir

La question

Nous savons ce que coûte chaque instruction. Nous pouvons donc, en principe, prédire exactement où en sera le faisceau au terme de n'importe quelle suite d'instructions.

En principe. Ce chapitre est celui où la machine nous fait mentir — et il vaut la peine, parce que le mensonge est régulier, mesurable, et qu'il porte un nom.

Deux instructions pour fabriquer un chronomètre

Nouvelle instruction — INC (« INCrément ») : ajoute 1 au contenu d'un casier, sur place, sans passer par le registre A. `inc $d021` fait passer la couleur de fond à la suivante. Elle coûte 6 cycles : lire, ajouter, réécrire.

Avec `inc` et `jmp`, on peut écrire le programme le plus court et le plus révélateur du livre. Deux instructions : incrémenter la couleur de fond, recommencer.

Le compte est immédiat : 6 cycles pour l'`inc`, 3 pour le `jmp`, soit **9 cycles par changement de couleur**. Et 9 cycles, c'est 72 pixels. Chaque strie que vous allez voir à l'écran *est* une mesure : elle dit « ici, neuf battements se sont écoulés ». L'écran devient un chronographe, et le faisceau son stylet.

Regardez encore le nombre 63, la durée d'une ligne. **63 divisé par 9 fait exactement 7**. Autrement dit, si rien ne vient troubler notre boucle, chaque ligne contiendra exactement sept stries, et chaque ligne sera identique à la précédente : les stries s'empileront en **colonnes parfaitement verticales**. Toute déformation que nous verrons sera donc, à coup sûr, du temps volé.

L'expérience

```
!to "badline.prg", cbm

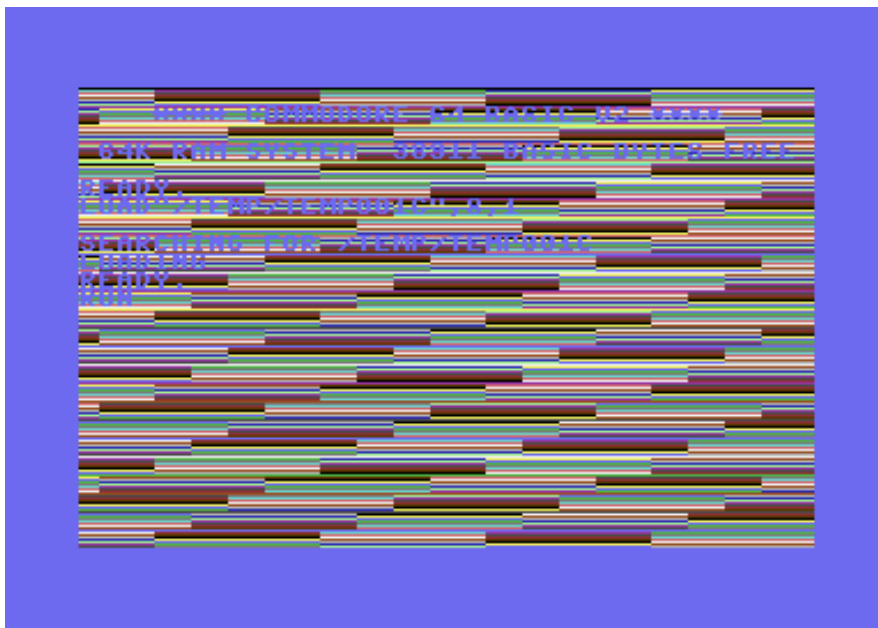
* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence

boucle inc $d021          ; 6 cycles : couleur de FOND + 1
      jmp boucle         ; 3 cycles -> 9 cycles par strie
                        ; 63 / 9 = 7 stries par ligne, pile
```

Nous peignons le **fond** et non la bordure, pour une raison précise : le fond est visible dans la fenêtre d'affichage, c'est-à-dire exactement là où le VIC travaille. C'est là qu'il faut installer notre chronomètre si nous voulons le voir se faire dévaliser.

Ce qu'on observe :



Capture réelle. Les colonnes sont là... et une ligne sur huit les traverse d'un seul bloc de couleur uniforme.

Voilà l'anomalie. Une ligne sur huit, notre chronomètre **s'arrête** : au lieu de ses sept changements de couleur, la ligne n'en montre presque aucun. Elle reste d'une seule teinte, sur toute sa largeur.

Ce n'est pas une impression — et ici je passe au « je », car ce qui suit n'est plus un raisonnement partagé mais une vérification que j'ai faite, machine allumée, et dont vous devez pouvoir douter. J'ai donc compté les changements de couleur, ligne par ligne, dans la capture ci-dessus : les lignes ordinaires en montrent 4 à 5 (une partie des stries tombe hors du cadre visible), et une ligne sur huit en montre **0,29 en moyenne**. Une sur huit, notre programme n'a pas eu le temps de faire quoi que ce soit.

Ce qui s'est passé dans le couloir

Souvenez-vous de l'atelier du chapitre 0 : un seul couloir vers la réserve, coupé en deux moitiés — une pour le VIC, une pour le processeur. Cet arrangement fonctionne pour les pixels : le VIC lit huit pixels d'un coup, ça lui suffit largement.

Mais toutes les huit lignes, le VIC doit faire autre chose. Il doit aller chercher la **ligne de texte suivante** : quels caractères afficher, et de quelle couleur. Quarante caractères, quarante couleurs. Et là, sa demi-microseconde par cycle ne suffit plus, même de très loin.

Alors il fait ce qu'un artiste sous contrat impitoyable est en droit de faire : il **lève la main**. Un signal (les électroniciens l'appellent BA, *Bus Available*) prévient le processeur trois cycles à l'avance. Le 6510 termine proprement le geste en cours, puis il se fige — pas « il ralentit » : il s'arrête, complètement. Le VIC prend alors le couloir **entier**, pour lui seul, pendant **40 à 43 cycles**. Sur les 63 que dure la ligne.

C'est ce que les programmeurs du C64 appellent une **Bad Line** — une « mauvaise ligne ». Le nom est injuste, d'ailleurs : elle n'a rien de mauvais, elle est le prix de l'affichage. Sans elle, l'écran serait vide.

Quand survient-elle ? Toutes les huit lignes, puisqu'un caractère fait huit pixels de haut : une fois la ligne de texte lue, le VIC a de quoi tenir huit lignes de balayage. Plus précisément, elle survient quand les **trois derniers bits du numéro de la ligne** tombent sur la valeur du **cran de défilement vertical** — un réglage à huit positions, rangé dans le casier \$d011, qui sert normalement à faire glisser l'image verticalement de zéro à sept pixels. Sa valeur par défaut est 3. Et de fait, dans la capture ci-dessus, les lignes assommées sont bien celles dont le numéro finit par 3.

Retenez cette dernière phrase. Le fait que la Bad Line dépende d'un registre que **nous pouvons écrire** est le secret le plus lourd de conséquences de toute la machine : c'est la porte d'entrée des chapitres 7 et 9.

La contre-épreuve

Une explication qui ne se laisse pas réfuter ne vaut rien. Si les Bad Lines existent parce que le VIC doit lire des caractères, alors **en éteignant l'affichage, elles doivent disparaître**.

Le casier \$d011 contient justement un interrupteur « écran allumé ». Sa valeur habituelle est \$1b ; en écrivant \$0b, on éteint l'affichage. (Ce casier porte huit interrupteurs indépendants et reviendra aux chapitres 7, 8 et 9 : l'annexe les donne un par un, avec les cinq valeurs que ce livre écrit dedans.) — le VIC n'a plus rien à lire, plus rien à afficher, et tout l'écran devient de la bordure.

```
!to "eteint.prg", cbm

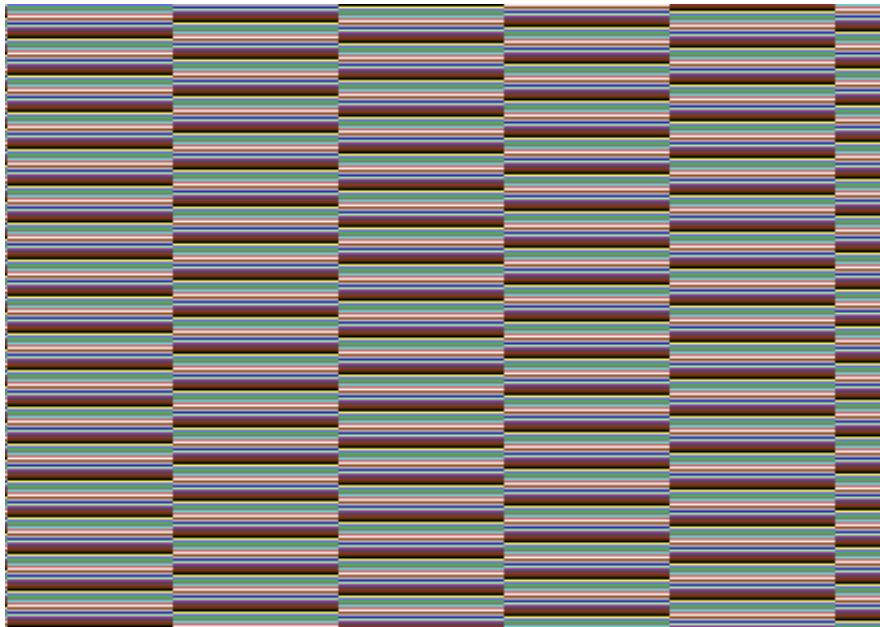
* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence
    lda #$0b           ; comme $1b, mais l'interrupteur
    sta $d011          ; « écran allumé » en moins

boucle inc $d020        ; 6 cycles
      jmp boucle        ; 3 cycles -> 9 cycles par strie
```

Un détail qui a son importance : nous revenons peindre la **bordure** et non le fond. Ce n'est pas une distraction — l'affichage étant éteint, il n'y a plus de fenêtre du tout : l'écran entier *est* de la bordure. Notre chronomètre gagne donc toute la surface.

Ce qu'on observe :



Capture réelle, écran éteint. Sept stries par ligne — six frontières visibles, la septième tombant hors du cadre capturé — alignées à la perfection sur les 272 lignes : plus une seule Bad Line.

Impeccable. J'ai vérifié à la machine : **exactement six changements de couleur par ligne** dans la fenêtre de capture, sur **toutes** les lignes sans exception, et un décalage de **zéro pixel** d'une ligne à la suivante. Nos colonnes verticales, exactement comme l'arithmétique les avait prédites.

La démonstration est faite : pas d'affichage, pas de lecture de caractères, pas de Bad Line. Le processeur récupère la totalité de ses 63 cycles par ligne. C'est aussi, soit dit en passant, la ruse que tous les programmeurs du C64 emploient quand ils ont un gros calcul à faire : on éteint l'écran, on calcule, on rallume. Le gain est modeste et il se calcule — vingt-cinq Bad Lines par image, quarante cycles chacune, mille cycles récupérés sur les 19 656 d'une image : **environ 5 %**, ou 8 % si l'on ne regarde que la zone d'affichage. C'est peu, mais c'est gratuit.

La patience, et une prédiction vérifiée au pixel

Reste une question de méthode : comment attend-on un nombre *choisi* de cycles ? Nos boucles de guet attendent une ligne ; il nous faudra bientôt attendre « douze cycles », pas plus.

Nouvelle instruction — DEX (« DEcrement X ») : retire 1 au registre X. Elle coûte 2 cycles, et ne touche à rien d'autre.

Associée à bne, elle donne le compte-à-rebours le plus économique de la machine :

```
ldx #2          ; 2 cycles : la duree de la patience
attente dex     ; 2 cycles a chaque passage
bne attente     ; 3 si on repart, 2 la derniere fois
```

Ajoutons cette patience à notre chronomètre — écran éteint, pour mesurer sans les Bad Lines :

```
!to "patience.prg", cbm

* = $0801          ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

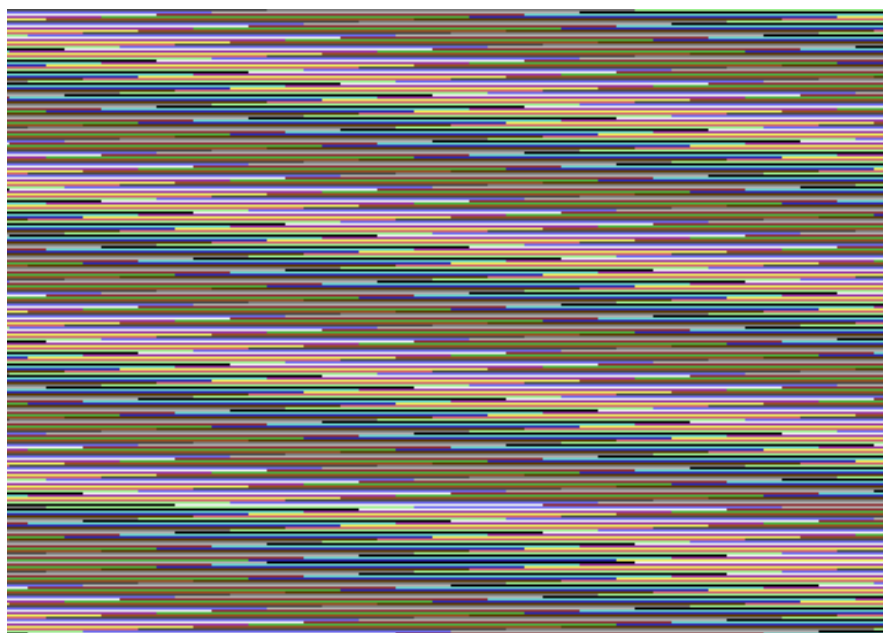
* = $0810
sei               ; silence
lda #$0b          ; ecran eteint : aucune Bad Line
sta $d011         ; ne viendra brouiller la mesure

boucle inc $d020   ; 6 cycles
ldx #2           ; 2 cycles : la duree de la patience
attente dex       ; 2 cycles a chaque passage
bne attente       ; 3 si on repart, 2 la derniere fois
jmp boucle        ; 3 cycles
; total : 6+2+(2+3)+(2+2)+3 = 20 cycles
```

Le coût par strie devient : 6 (inc) + 2 (ldx) + 5 (premier passage, branchement pris), puis 4 (dernier passage) et 3 (jmp) = **20 cycles**.

Et 20 ne divise pas 63. C'est même tout l'intérêt : $63 = 3 \times 20 + 3$. Chaque ligne se termine donc 3 cycles « en avance » sur le motif, et les stries de la ligne suivante seront décalées de 3 cycles — c'est-à-dire, à huit pixels le cycle, de **24 pixels**. Nos colonnes verticales doivent devenir un escalier de pente exactement 24 pixels par ligne.

Ce qu'on observe :



Capture réelle. La prédiction était : 24 pixels de décalage par ligne. Mesuré dans l'image : 24 pixels, vers la gauche, sur toute la hauteur.

J'ai fait la mesure plutôt que de vous demander de me croire : le décalage médian entre deux lignes consécutives vaut **exactement** —**24 pixels**, et le nombre de stries par ligne est tombé à 2 ou 3 (20 cycles font 160 pixels, il en tient deux et demi dans la largeur visible). L'arithmétique du chapitre 3 a prédit l'image au pixel près.

C'est le moment de mesurer ce que nous venons d'acquérir. Nous savons compter les cycles, nous savons en dépenser un nombre choisi, et nous savons que le VIC nous en volera 40 toutes les huit lignes. Nous ne sommes plus spectateurs du temps : nous le tenons.

Sous le capot — la Bad Line a une définition exacte, à trois conditions simultanées (fenêtre de lignes, égalité avec le cran de défilement, affichage allumé au bon moment), et le gel du processeur passe par deux signaux distincts (BA puis AEC) dont le décalage de trois cycles aura une conséquence spectaculaire au chapitre 9. Tout cela est spécifié dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §5 (« Bad Lines — définition exacte ») ; les signaux de bus BA et AEC sont au §1 du même chapitre, et le décompte cycle par cycle du vol au chapitre 2.

Au prochain chapitre

Le VIC vient de voler quarante cycles pour aller lire « la ligne de texte suivante ». Nous avons parlé de ces caractères sans jamais les regarder. D'où viennent-ils ? Où sont-ils rangés ? Et si nous écrivions dedans nous-mêmes ?

Chapitre 5 — Les cellules : d'où viennent les caractères

La question

Avant même que vous ayez tapé quoi que ce soit, il y a du texte à l'écran : le message d'accueil, `READY.`, le curseur qui bat. Quelqu'un a bien dû l'écrire. Et au chapitre 4, nous avons vu le VIC geler le processeur une quarantaine de cycles, toutes les huit lignes, pour aller chercher quarante octets. Quarante octets **de quoi**, exactement, et **où** ?

La réponse est d'une simplicité presque décevante : les caractères de l'écran sont **mille octets de mémoire ordinaire**, dans la réserve, comme les autres. Écrivez un nombre dans l'un d'eux, un caractère apparaît — pas d'appel système, pas de `PRINT` : un `sta`. Et à la fin du chapitre, nous déménagerons l'écran entier ailleurs en mémoire en écrivant **un seul octet**.

L'expérience — mille casiers

Le premier casier de l'écran porte le numéro 1024, que nous écrirons `$0400`. Il y en a mille à la suite, jusqu'à 2023 (`$07e7`). Voici comment on les remplit tous.

```

; -----
; 20 MILLISECONDES – chapitre 5 : la matrice vidéo, à la main
;
; Mille casiers à partir de $0400 : un octet déposé = un
; caractère affiché. Aucun PRINT, aucun appel au système.
; On peint l'écran entier, puis on écrit trois lettres à la main
; dans le coin haut-gauche.
;
; Assembler :   acme matrice.a
; Lancer :      LOAD"MATRICE",8,1 puis RUN
;               (RUN/STOP + RESTORE pour revenir au BASIC)
; -----
!to "matrice.prg", cbm

* = $0801                ; l'amorce du chapitre 1 : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
sei                      ; le curseur du BASIC ne viendra pas clignoter dessus

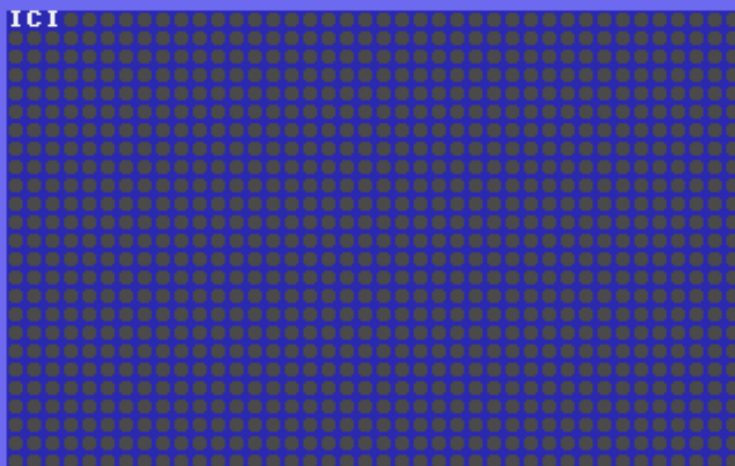
; --- les 1000 cellules, en quatre passes de 256 ---
ldx #0
rempli lda #81           ; code ÉCRAN 81 : le disque plein
      sta $0400,x        ; cellules 0 à 255
      sta $0500,x        ; cellules 256 à 511
      sta $0600,x        ; cellules 512 à 767
      sta $06e8,x        ; cellules 744 à 999 (24 de recouvrement)
      lda #11           ; 11 : gris foncé
      sta $d800,x        ; la couleur de la cellule 0, puis 1, puis 2...
      sta $d900,x
      sta $da00,x
      sta $dae8,x
      dex
      bne rempli

; --- trois octets à la main : le coin haut-gauche, c'est $0400 ---
      lda #9             ; I
      sta $0400
      lda #3             ; C
      sta $0401
      lda #9             ; I
      sta $0402
      lda #1             ; blanc
      sta $d800          ; ...pour ces trois cellules-là seulement
      sta $d801
      sta $d802

fini   jmp fini          ; on s'arrête ici : l'image reste telle quelle

```

Ce qu'on observe :



Capture réelle (C64 Ultimate, sortie vidéo native).

Le message d'accueil a disparu, recouvert : l'écran entier — vingt-cinq rangées, quarante colonnes — est pavé du même disque plein, gris foncé sur fond bleu. Et dans le coin haut-gauche, trois lettres blanches, **ICI**, qui désignent le casier \$0400 : le premier octet de la matrice s'affiche là, le suivant \$0401 juste à sa droite, le quarantième termine la première rangée, le quarante-et-unième commence la deuxième. Mille casiers lus à la file, comme on lit une page — et pas une instruction nouvelle pour ça : ce chapitre apporte une **carte**.

La **matrice vidéo** est un tableau de **1000 octets** en mémoire ordinaire, à partir de \$0400 (1024) par défaut. L'octet numéro n décide quel caractère s'affiche dans la cellule numéro n , en partant du coin haut-gauche, de gauche à droite puis de haut en bas. 25 rangées $\times$ 40 colonnes = 1000.

Rien d'autre : pas de commande d'affichage, pas de protocole. Le VIC lit ces mille casiers et dessine ce qu'il y trouve, cinquante fois par seconde, que vous les ayez remplis ou non.

L'élégance du vieux routier : 1000 casiers en quatre passes de 256

Notre boucle mérite un mot, parce qu'elle est un classique absolu et que sa petite bizarrerie est délibérée.

Un registre d'index tient un nombre de 0 à 255 : une passe de boucle ne peut donc couvrir que 256 casiers. Mille casiers, ça fait quatre passes — mais $4 \times 256 = 1024$, soit **24 casiers de trop**. Si le quatrième bloc partait de \$0700, il déborderait de 24 casiers *après* la fin de l'écran (\$07e8–\$07ff), c'est-à-dire précisément là où le VIC ira chercher autre chose au chapitre suivant. On fait donc démarrer le quatrième bloc 24 casiers **plus tôt**, en \$06e8 : ses 256 écritures couvrent les cellules 744 à 999, la dernière tombant pile sur la dernière cellule de l'écran. Les 24 écritures en trop retombent sur des cellules déjà peintes, avec exactement la même valeur : rien n'est abîmé, rien ne dépasse.

Vérifions, parce que ce livre ne demande jamais de croire : $\$06e8 = 1768$, et $1768 - 1024 = 744$; $744 + 256 = 1000$. Le compte est bon.

Deux détails de la boucle valent la peine qu'on s'arrête :

- **Pourquoi deux lda par tour ?** Parce que le registre A ne tient qu'une valeur à la fois. Il faut le code du caractère pour les quatre sta de la matrice, puis la couleur pour les quatre suivants. A est un seul seau : on le remplit deux fois par tour.
- **Pourquoi ldx #0 et non ldx #255 ?** Parce que dex fait passer X de 0 à 255 (il « repasse par en dessous ») : le premier tour traite la cellule 0, puis 255, 254... jusqu'à 1, où bne cesse de brancher. Les 256 valeurs y passent, dans un ordre bizarre et sans aucune importance.

Les codes écran ne sont PAS les codes ASCII

C'est ici que tout le monde trébuche. Dans la matrice, la lettre A ne s'écrit pas 65, elle s'écrit 1 ; le B vaut 2, le C vaut 3... le Z vaut 26 ; l'espace vaut 32, les chiffres 0 à 9 valent 48 à 57 ; le code 0 donne @ et les codes 64 à 127 des symboles graphiques, que n'importe quel code + 128 rend en vidéo inverse. Notre mot « ICI » s'écrit donc 9, 3, 9. Le 65 existe aussi, mais c'est le code du BASIC, celui de CHR\$(65) et de ASC("A") : déposé dans la matrice il **n'affiche pas un A**, puisque les codes 65 à 90 y sont des graphismes. Deux tables, deux mondes.

Pourquoi cette double vie ? Parce que le nombre déposé n'est pas une lettre, c'est un **numéro de tiroir** : le dessin des caractères vit dans le **générateur de caractères**, un meuble de 256 tiroirs de 8 octets (un octet par ligne de pixels, 8 lignes par cellule), où le VIC va lire *base du générateur + 8 $\times$ code + numéro de ligne dans la cellule*.

La Color RAM : mille casiers de plus, et un piège

Chaque cellule a **sa** couleur, et cette couleur vit dans une mémoire séparée : mille autres casiers à partir de \$d800 (55296), la cellule n prenant la couleur rangée dans $\$d800 + n$. Même numérotation, décalage constant — c'est pour ça que notre boucle peint les deux tableaux dans le même tour, avec le même X. Deux particularités : seuls les **quatre bits de poids faible** de chaque casier servent (une couleur de 0 à 15 ; cette mémoire n'est large que de 4 bits), et elle **ne peut pas bouger** — \$d800–\$dbe7, définitivement. Retenez-le : dans quelques pages nous déplacerons l'écran, et elle restera là.

Et le piège, qui a fait perdre du temps à des générations de débutants : **un caractère déposé sans sa couleur peut être parfaitement invisible**. La routine d'effacement d'écran recopie dans les mille casiers de couleur la **couleur de fond** du moment ; un caractère écrit ensuite dans la matrice, sans toucher à la Color RAM, est dessiné en bleu sur bleu — il est bien là, le VIC l'affiche fidèlement, et vous ne voyez rien. La seule façon de savoir de quelle couleur sortira votre caractère, c'est **de l'écrire vous-même**.

Ces mille octets, c'est le butin de la Bad Line

Les *dessins*, le VIC les lit à chaque ligne de balayage : 40 lectures dans le générateur de caractères, une par colonne (les accès « g »). Mais les *codes* ne changent pas d'une ligne de pixels à l'autre, alors il ne les lit qu'**une fois par rangée de texte** — et ces 40 lectures (les accès « c ») n'ont pas la place de tenir dans sa demi-microseconde. Voilà ce qu'est la Bad Line du chapitre 4 : le processeur gelé une quarantaine de cycles, et le VIC qui emporte **les 40 octets de la rangée en cours**. Le vol est double, d'ailleurs : chaque accès « c » ramène **12 bits** d'un coup, 8 bits de la matrice et 4 bits de la Color RAM, par un bus élargi exprès pour ça. Et il ne se répète que huit lignes sur huit : entre deux Bad Lines, le VIC travaille sur une **copie interne** de quarante codes et quarante couleurs, si bien qu'un caractère déposé dans la matrice n'existe pour lui qu'à la prochaine Bad Line de sa rangée. À l'œil, c'est instantané ; retenez quand même la phrase, les derniers chapitres en feront un instrument.

Deuxième expérience — l'écran n'est qu'une adresse

Question naïve, et pourtant : *pourquoi \$0400* ? Qu'est-ce qui, dans cette machine, sait que l'écran est là ? Un registre, un seul : \$d018. Le VIC y lit deux réponses.

Bits de \$d018	Ce qu'ils désignent	Pas
les 4 bits du haut	où le VIC lit la matrice vidéo	1 Ko (1024 octets)
les bits 3, 2 et 1	où il lit le générateur de caractères	2 Ko
le bit 0	rien, il ne sert pas	—

Les quatre bits du haut ne prennent que seize valeurs — la matrice ne peut donc se poser que sur un multiple de 1024 — et comme ils occupent le **haut** de l'octet, le numéro de matrice doit être multiplié par 16 : la matrice n° 1 (\$0400, celle du démarrage) contribue $1 \times 16 = 16$ au registre, la n° 8 (\$2000, celle que nous allons fabriquer) $8 \times 16 = 128$. Les trois bits du milieu se comptent de la même façon, et la valeur du démarrage vaut 4 : le générateur de caractères de la ROM, que le VIC voit en \$1000. Nous n'y toucherons pas — nous voulons de vraies lettres, pas nos propres dessins (ce sera un autre livre, ou votre prochain week-end).

Où poser la deuxième matrice ? (une décision, pas une recette)

Il faut mille octets libres sur un multiple de 1024. Trois candidats se présentent, et deux se disqualifient tout seuls :

- **\$0800** — le voisin immédiat, tentant. **Non** : c'est là que le BASIC range le texte des programmes (\$0800-\$9fff), et notre programme *est* un programme BASIC, ou du moins il en a l'amorce : il vit en \$0810. Y installer la matrice, ce serait écrire mille octets par-dessus le code en train de les écrire. Suicide en pleine boucle.
- **\$1000-\$1fff** — libre en apparence, et piégé : dans la banque de mémoire où le VIC travaille par défaut, c'est là qu'il voit l'image de la ROM des caractères. Il y lirait des dessins en croyant lire des codes.
- **\$2000** — 8192, multiple de 1024, RAM franche, loin de notre code et loin de la ROM des caractères. C'est notre adresse. (Elle est d'ailleurs le choix traditionnel pour ce genre de besoin.)

Voilà le genre d'arbitrage que la machine impose en permanence, et qu'aucun manuel ne fera à votre place : trois contraintes, une adresse qui les satisfait.

Une lettre par cellule, à des places choisies

Notre deuxième matrice ne se contentera pas d'un pavage uniforme : nous y poserons sept lettres en diagonale, chacune à un numéro de cellule pris dans une table. Un registre de plus, donc.

Nouveau registre et nouvelle instruction — LDY (« LoaD Y ») : charge un octet dans le registre Y, le troisième et dernier registre de travail du 6510, après A et X. Y sert d'index exactement comme X : sta \$2000,y = « dépose A dans le casier situé Y cases après \$2000 ». Pourquoi un deuxième ? Parce qu'un registre ne fait qu'une chose à la fois. Dans la boucle qui suit, **X compte** (c'est lui que dex fait décroître, c'est lui qui décide quand on s'arrête) ; il ne peut pas *en même temps* désigner la cellule de destination, qui saute de 41 en 41. Deux rôles, deux registres : X compte, Y désigne.

Notez la forme ldy places,x : le mode indexé du chapitre 3, appliqué à Y. C'est le programme qui décide de la position **pendant** qu'il tourne.

```
;-----
; 20 MILLISECONDES – chapitre 5 : l'écran déménage
;
; On bâtit une SECONDE matrice vidéo en $2000, invisible.
; Puis une seule écriture dans $d018 dit au VIC d'aller lire
; là-bas : l'écran change d'un coup, sans qu'un seul caractère
; affiché ait été réécrit.
;
; Assembler :   acme demenage.a
; Lancer :      LOAD"DEMENAGE",8,1 puis RUN
;              (RUN/STOP + RESTORE pour revenir au BASIC)
;-----
!to "demenage.prg", cbm

* = $0801                ; l'amorce du chapitre 1 : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; personne ne vient nous déranger

; --- 1. la deuxième matrice, en $2000 : personne ne la regarde encore ---
    ldx #0
mur   lda #81           ; le même disque plein qu'au listing précédent
      sta $2000,x       ; cellules 0 à 255
      sta $2100,x       ; cellules 256 à 511
      sta $2200,x       ; cellules 512 à 767
      sta $22e8,x       ; cellules 744 à 999
      lda #11          ; gris foncé
      sta $d800,x       ; la couleur, elle, reste où elle a toujours été
      sta $d900,x
      sta $da00,x
      sta $dae8,x
      dex
      bne mur

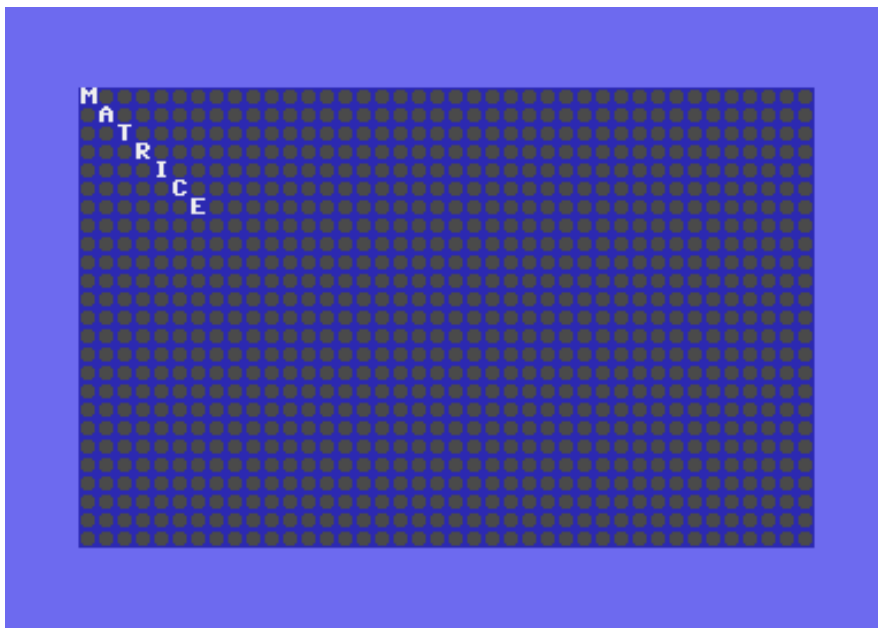
; --- 2. sept lettres, chacune à sa place : X compte, Y désigne ---
    ldx #7
mot   ldy places,x      ; Y <- le numéro de cellule où va la lettre
      lda lettres,x     ; A <- son code écran
      sta $2000,y       ; la lettre, dans la matrice n° 2...
      lda #1           ; blanc
      sta $d800,y       ; ...sa couleur, dans la Color RAM (qui n'a pas bougé)
      dex
      bne mot

; --- 3. une seule écriture, et le VIC va lire ailleurs ---
      lda #132         ; 128 = matrice en $2000 ; 4 = caractères en $1000
      sta $d018

fini  jmp fini          ; on s'arrête ici : l'image reste telle quelle

; La case 0 des deux tables n'est jamais lue : la boucle s'arrête à X=1.
places !byte 0,  0, 41, 82, 123, 164, 205, 246  ; une diagonale, une cellule par ligne
lettres !byte 0, 13,  1, 20, 18,  9,  3,  5  ; M A T R I C E
```

Ce qu'on observe :



Capture réelle (C64 Ultimate, sortie vidéo native).

Le même mur de disques gris — mais il ne vient plus du même endroit. Et en diagonale depuis le coin haut-gauche, une cellule plus bas et une plus à droite à chaque fois, les sept lettres blanches : **M A T R I C E**. Pendant tout le remplissage, l'écran affichait encore le message d'accueil du BASIC : nous écrivions en \$2000, et personne ne regardait \$2000. Puis deux instructions, `lda #132` et `sta $d018`, et l'image a changé entièrement. **Pas un seul code de caractère affiché n'a été réécrit** : nous n'avons pas déplacé mille octets, nous avons déplacé le regard du VIC.

L'explication — $132 = 128 + 4$

Le nombre déposé dans \$d018 assemble deux réponses : **128** dans les quatre bits du haut (« la matrice est en \$2000 ») et **4** dans les bits 3-2-1 (« le générateur de caractères reste celui de la ROM, en \$1000 »). Ce registre porte deux informations sans rapport et l'on ne veut déplacer que l'une des deux : c'est toute la précaution à prendre avec lui. Détail savoureux au passage, la documentation Commodore recommande `POKE 53272, 21` pour revenir à la normale là où le système d'exploitation écrit **20** : les deux ont raison, car le bit 0 n'est pas branché et se lit toujours à 1. La valeur qu'on écrit et celle qu'on relit ne sont pas forcément la même.

Regardez maintenant les deux blocs de `sta` de la première boucle : les codes partent vers \$2000, les couleurs vers \$d800. Rien ne les suit, et les mille couleurs sont donc **partagées** par toutes les matrices que vous installerez — la cellule numéro 246 prendra la couleur du casier \$d800 + 246, que ses codes viennent de \$0400 ou de \$2000. C'est pourquoi le message d'accueil a changé de couleur pendant notre remplissage, et c'est ce qui explique la ligne la plus étrange du listing :

```
sta $2000,y      ; la lettre, dans la matrice n° 2...
lda #1           ; blanc
sta $d800,y      ; ...sa couleur, dans la Color RAM (qui n'a pas bougé)
```

Deux `sta`, le même Y, deux régions de mémoire qui n'ont rien à voir — et une seule cellule à l'écran. Vous venez d'écrire, en assembleur, la routine d'affichage du Commodore 64.

Une honnêteté pour finir. Quand notre `sta $d018` s'exécute, le faisceau est au milieu de l'image, et les rangées au-dessus de lui ont déjà eu leur Bad Line : leurs quarante codes viennent de l'**ancienne** matrice. Le basculement descend donc l'écran **rangée de texte par rangée de texte**, au rythme des Bad Lines, et il est terminé au bout d'une image : vingt millisecondes. C'est pour ça que l'œil ne voit qu'un changement net — et c'est pour ça qu'un programme qui écrirait \$d018 plusieurs fois *dans la même image* afficherait **deux matrices différentes sur le même écran**, l'une au-dessus, l'autre en dessous. Le réussir demande de savoir où est le faisceau au cycle près : c'est le programme des chapitres 7 et 9.

Sous le capot — tout ce chapitre est la version douce de quatre sections de *Au cœur du métal* — *Commodore 64 & Ultimate 64*, chapitre 1 : la matrice vidéo et les deux sortes d'accès du VIC (§6.2),

les compteurs qui la parcourent et la règle des 40 lectures pendant les cycles 15 à 54 (§8), les formules d'adresse exactes des accès c et g (§9), et $\$d018$ bit par bit (§2). Les codes écran et l'interdiction faite à la Color RAM de bouger viennent, eux, du *Commodore 64 Programmer's Reference Guide* (appendice B, et le chapitre « Programming Graphics »).

Au prochain chapitre

Il reste huit octets dont nous n'avons rien dit : ceux qui suivent la fin de la matrice — $\$07f8$ – $\$07ff$ quand l'écran est à sa place habituelle, $\$23f8$ – $\$23ff$ depuis notre déménagement — et que nous avons évité de piétiner avec notre quatrième bloc de 256 écritures. Ils désignent des dessins que le VIC va chercher tout seul, huit fois par ligne de balayage, et qu'il affiche **par-dessus** la grille des cellules, n'importe où, au pixel près. Au chapitre 6 : les sprites.

Chapitre 6 — Les sprites : huit objets libres

La question

Depuis le début de ce livre, tout ce que vous dessinez est prisonnier d'une grille : un caractère ne peut se poser que *dans* une cellule, et pour déplacer une figure d'un pixel il faudrait redessiner les caractères qu'elle occupe, puis effacer le décor derrière elle. Aucun jeu ne fonctionne comme ça. Alors comment le Commodore 64 promène-t-il un vaisseau, une balle, un monstre au pixel près, au-dessus d'un décor qu'il n'abîme pas ?

La réponse du VIC-II est d'une générosité surprenante : à côté de sa machine à caractères, il embarque **huit petits dessinateurs indépendants**. MOS les appelait des *MOBs* — *Movable Object Blocks* ; tout le monde dit **sprites**. Chacun porte un dessin de **24 pixels de large sur 21 de haut**, se place où vous voulez **au pixel près**, possède sa propre couleur, passe par-dessus l'image sans y toucher, et ne consomme pas un octet de décor. Gratuits ? Non : ils se paient en **cycles**, et c'est la troisième des trois expériences de ce chapitre — faire apparaître une créature, en aligner huit, les regarder voler du temps.

C'est le chapitre le plus long du livre, et ses trois expériences se suffisent chacune à elle-même : si vous ne devez en lancer qu'une aujourd'hui, prenez la première.

Le dessin : vingt-et-une lignes de 0 et de 1

Vingt-quatre pixels de large, un octet de huit bits : **trois octets par ligne de sprite**, et vingt-et-une lignes de trois octets font **63 octets** — la taille exacte d'un dessin de sprite, toujours. Les trois octets se lisent de gauche à droite, et dans chaque octet le bit de poids fort est le pixel de gauche. Un bit à **1** = un pixel de la couleur du sprite ; un bit à **0** = **rien du tout**, transparent, on voit l'écran à travers — c'est ce qui donne aux sprites leur silhouette. En hexadécimal, ces 63 octets seraient illisibles ; heureusement, l'assembleur sait lire les nombres bit par bit.

Nouvelle notation — le binaire % : %01111110 désigne un octet écrit **bit par bit**, huit chiffres, un par bit. C'est le même nombre que 126 ou que \$7e — la même valeur, une autre écriture. Le chiffre le plus à gauche est le **bit 7** (le poids fort), celui le plus à droite le **bit 0**. Pour un dessin de sprite, c'est exactement l'ordre des pixels à l'écran : gauche à droite. Vous ne codez plus, vous dessinez.

Et cela donne un dessin qu'on lit comme un dessin, dans le source même du programme :

```
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
```

Ces huit lignes sont le haut d'une petite créature ronde : les cinq premières dessinent un crâne qui s'élargit, puis deux trous de trois 0 creusés dans les 1, un de chaque côté — les yeux. Le corps et une large bouche ouverte suivent la même logique, vous les lirez dans le listing. Prenez du papier quadrillé, 24 cases sur 21, noircissez ce que vous voulez, recopiez ligne par ligne : c'est tout le métier, celui des jeux que vous avez aimés y compris.

L'expérience — un sprite, immobile, au centre

Le VIC ne pose que quatre questions sur un sprite — **où est son dessin, où sur l'écran, de quelle couleur, est-il allumé** — et il y a un casier-levier pour chacune.

Où est le dessin ? Le casier \$07f8. Il ne contient pas une adresse mais un **numéro de bloc de 64 octets** : nous rangeons notre dessin à l'adresse 2560 (\$0a00), qui vaut 40×64 , et nous écrivons 40 dans \$07f8. La raison de cette division vient juste après l'expérience.

Où sur l'écran ? \$d000 pour le X du sprite 0, \$d001 pour son Y. L'écran fait 320 pixels de large et un octet ne compte que jusqu'à 255 : le X des sprites a donc un **neuvième bit**, rangé à part dans \$d010, un par sprite. Nous voulons X = 172, qui place les 24 pixels du dessin pile au milieu de la fenêtre d'affichage ; la valeur tient dans un octet, donc le neuvième bit du sprite 0 doit être **éteint**. Quant au Y, il réserve une surprise dont il faut se souvenir : **le VIC compare la coordonnée Y à la fin de la ligne précédente**, si bien que le dessin ne commence qu'à la ligne *suivante*. Écrivez 140, la première ligne dessinée est la 141.

De quelle couleur ? \$d027 pour le sprite 0, une seule couleur pour tout le dessin (le VIC sait faire mieux, nous n'en aurons pas besoin ici). **Est-il allumé ?** \$d015 : un casier, huit interrupteurs, le bit 0 pour le sprite 0, le bit 7 pour le sprite 7 — et c'est ici qu'il nous manque deux instructions, car nous voulons allumer **un** interrupteur sans toucher aux sept autres.

Nouvelle instruction — ORA (« OR with Accumulator ») : compare bit par bit le registre A et un octet, et garde un 1 partout où **au moins l'un des deux** avait un 1. `ora #00000001` **allume** le bit 0 de A et laisse les sept autres exactement comme ils étaient. Le motif complet — lire le casier, allumer un bit, réécrire — est le geste quotidien du programmeur C64 : `lda $d015 / ora #00000001 / sta $d015`.

Nouvelle instruction — AND (« AND with Accumulator ») : compare bit par bit le registre A et un octet, et ne garde un 1 que là où **les deux** avaient un 1. Autrement dit, tout bit en face d'un 0 est **éteint**, tout bit en face d'un 1 est **préservé**. `and #11111110` éteint le bit 0 et ne touche à rien d'autre. ORA allume, AND éteint : avec ces deux-là, vous êtes maître d'un interrupteur sur huit.

```

;-----
; 20 MILLISECONDES – chapitre 6 : un objet libre
;
; Une créature de 24 x 21 pixels, immobile au centre de l'écran.
; Elle flotte AU-DESSUS du texte : le BASIC reprend la main, vous
; pouvez taper, l'écran défile – elle ne bouge pas d'un pixel.
;
; Assembler :   acme unsprite.a
; Lancer :       LOAD"UNSPRITE",8,1 puis RUN
;-----
!to "unsprite.prg", cbm

* = $0801                ; l'amorce du chapitre 1 : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
; --- où est le dessin ? bloc n° 40, car 40 x 64 = 2560 = $0a00
lda #40
sta $07f8                ; le casier-pointeur du sprite 0

; --- où est le sprite ?
lda #172                 ; X = 172 : le milieu de la fenêtre
sta $d000
lda $d010                ; les huit bits « ce X dépasse 255 »
and #%11111110          ; on ÉTEINT celui du sprite 0, les sept autres intacts
sta $d010
lda #140                 ; Y = 140 : la créature commence ligne 141
sta $d001

; --- de quelle couleur ?
lda #7                   ; jaune
sta $d027

; --- et on l'allume
lda $d015                ; les huit interrupteurs d'activation
ora #%00000001          ; on ALLUME celui du sprite 0, les sept autres intacts
sta $d015

rts                      ; le BASIC reprend la main – la créature reste

;-----
; Le dessin : 21 lignes de 3 octets = 63 octets.
; Adresse OBLIGATOIREMENT multiple de 64 ($0a00 = 40 x 64).
; Un « 1 » = un pixel de la couleur du sprite, un « 0 » = rien
; du tout : on voit l'écran à travers.
;-----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

Ce qu'on observe :



Quinze instructions, et une créature jaune se tient au milieu de l'écran. Le rts a rendu la main au BASIC : le READY. est revenu, le curseur clignote, la machine est à vous — et la créature est toujours là. Faites l'essai qui compte : **tapez**. Le texte passe **derrière** elle, glisse sous elle, disparaît en haut ; elle ne bouge pas, ne s'efface pas, ne se déchire pas. Elle n'est pas *dans* l'image : elle est *devant*.

L'explication — pourquoi 64 ?

Pourquoi le pointeur compte-t-il en blocs de 64 octets au lieu de donner une adresse ? Parce qu'il ne fait **qu'un octet**, de 0 à 255, là où une adresse en demanderait deux : le VIC ne veut pas savoir *où*, il veut savoir *lequel*. Et le compte tombe rond, car il n'a que quatorze fils d'adresse et ne voit donc que **16 kilo-octets** à la fois — une « banque », celle qui commence à l'adresse 0 tant qu'on n'y touche pas. Or **256 blocs × 64 octets = 16 384 octets**, exactement ces 16 kilo-octets. À retenir : le pointeur, c'est **l'adresse du dessin divisée par 64**, et l'adresse doit donc être un **multiple de 64**. $2560 \div 64 = 40$. (Un dessin fait 63 octets et un bloc 64 : le dernier octet est perdu, c'est le prix des nombres ronds.)

Et pourquoi \$07f8 ? Parce que les huit pointeurs sont rangés **juste derrière l'écran** : la matrice vidéo du chapitre 5 occupe mille casiers à partir de \$0400, et les huit derniers octets du kilo-octet qui la contient — \$07f8 à \$07ff — sont les pointeurs des sprites 0 à 7. Ils suivent l'écran comme son ombre : déplacez la matrice, les pointeurs la suivent.

Reste le mécanisme d'affichage, qui explique tout le reste du chapitre. À **chaque ligne de balayage**, le VIC lit le pointeur du sprite puis, si le sprite est visible sur cette ligne, ses **trois octets** de la ligne en cours ; il les charge dans un **registre à décalage de 24 bits** et, dès que le faisceau atteint la coordonnée X du sprite, il pousse ce registre vers la gauche, **un bit par pixel** — le premier bit sorti est le bit 7 du premier octet, le pixel de gauche. Deux conséquences gouvernent tous les jeux du Commodore 64 : après 24 pixels le registre est **vide**, un sprite ne peut donc pas servir deux fois sur la même ligne (**huit sprites par ligne, pas neuf**) ; mais plus bas dans l'image, un sprite qui a fini son affichage **peut resservir**, il suffit de changer son Y. C'est ainsi que des jeux affichent vingt objets avec huit sprites, au prix d'un chronométrage serré.

L'expérience — les huit, en rang

Puisqu'il y en a huit, alignons-les. La géographie des casiers est d'une régularité mécanique : les positions vont par paires de \$d000 à \$d00f (**pair = X, impair = Y**), les couleurs de \$d027 à \$d02e, les pointeurs de \$07f8 à \$07ff, un interrupteur chacun dans \$d015, un neuvième bit de X chacun dans \$d010. Le listing qui suit écrit ses trente-quatre valeurs une à une, sans boucle ni table indexée : c'est long à lire, mais rien n'y est caché. Une seule finesse, la rangée centrée met le huitième sprite à X = 284, au-delà de 255 — son neuvième bit entre donc en scène, et lui seul.

```

;-----
; 20 MILLISECONDES – chapitre 6 : les huit, en rang
;
; Les huit sprites, même dessin, huit couleurs, alignés au milieu
; de l'écran. Aucune astuce : chaque position est écrite à la main,
; casier par casier. C'est long à lire, mais rien n'est caché.
;
; Assembler :   acme huit.a
; Lancer :       LOAD"HUIT",8,1 puis RUN
;-----
!to "huit.prg", cbm

* = $0801                ; l'amorce du chapitre 1 : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    lda #dessin / 64      ; l'assembleur fait la division : 2560 / 64 = 40
    sta $07f8             ; les huit pointeurs, un par sprite...
    sta $07f9
    sta $07fa
    sta $07fb
    sta $07fc
    sta $07fd
    sta $07fe
    sta $07ff             ; ...tous sur le MÊME dessin

    lda #140              ; même Y pour les huit : la même rangée
    sta $d001
    sta $d003
    sta $d005
    sta $d007
    sta $d009
    sta $d00b
    sta $d00d
    sta $d00f

    lda #60               ; les X, un par un, de 32 pixels en 32 pixels
    sta $d000
    lda #92
    sta $d002
    lda #124
    sta $d004
    lda #156
    sta $d006
    lda #188
    sta $d008
    lda #220
    sta $d00a
    lda #252
    sta $d00c
    lda #28               ; le huitième : 284 = 256 + 28
    sta $d00e
    lda #%10000000        ; ...son 9e bit, et lui seul
    sta $d010

```

(Les huit, en rang : voici la **suite du même fichier**, à coller directement après la première partie. Les deux moitiés ne font qu'un seul programme.)

```

lda #1                ; huit couleurs, une par sprite
sta $d027             ; blanc
lda #7
sta $d028             ; jaune
lda #8
sta $d029             ; orange
lda #10
sta $d02a             ; rouge clair
lda #13
sta $d02b             ; vert clair
lda #3
sta $d02c             ; cyan
lda #14
sta $d02d             ; bleu clair
lda #4
sta $d02e             ; rose

lda #%11111111        ; les huit interrupteurs d'un coup
sta $d015
rts

; -----
; Le dessin : 21 lignes de 3 octets = 63 octets.
; Adresse OBLIGATOIREMENT multiple de 64 ($0a00 = 40 x 64).
; Un « 1 » = un pixel de la couleur du sprite, un « 0 » = rien
; du tout : on voit l'écran à travers.
; -----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

Ce qu'on observe :



Huit créatures, huit couleurs, une rangée bien droite en travers de l'écran. Le pointeur est le même pour les huit : **un seul dessin en mémoire, huit exemplaires à l'écran**, chaque sprite n'emportant que ses coordonnées, sa couleur et son interrupteur. Remarquez aussi ce qui a disparu : plus de ora, plus de and. Quand on écrit les huit bits d'un coup — `lda #%11111111` puis `sta $d015` — aucun masque n'est nécessaire ; ORA et AND servent quand on veut n'en changer **qu'un** en respectant les autres, ce qui est presque toujours le cas dans un vrai programme.

Deux règles de cohabitation avant de les faire bouger. En cas de chevauchement, **le sprite de plus petit numéro passe devant** : la hiérarchie est câblée, le sprite 0 gagne toujours, le sprite 7 perd toujours. Et **la bordure gagne contre tout le monde** — un sprite qui y entre disparaît dessous, sauf si on l'ouvre, ce qui est un chapitre entier à lui seul (le 8, où nous ouvrirons celle du haut et du bas ; les bordures latérales exigent un travail au cycle près sur chaque ligne et sortent du cadre de ce livre).

L'expérience — les huit voleurs

Maintenant, la facture. Reprenons le chronomètre visuel du chapitre 4 : une boucle qui ne fait qu'incrémenter une couleur, de sorte que **chaque écriture laisse une frontière de couleur** à l'endroit précis où le faisceau se trouvait — là où le processeur avance normalement, ces frontières forment des colonnes bien droites ; là où on lui vole des cycles, elles se déplacent. Deux réglages, cette fois. On peint **aussi le fond** (`$d021`), pour que les stries traversent la zone des sprites. Et on cale la boucle sur **21 cycles** : `inc` sur un casier absolu coûte 6 cycles, `jmp` en coûte 3, donc $6 + 6 + 6 + 3 = 21$. Or $21 \times 3 = 63$, la longueur exacte d'une ligne de balayage en PAL : le motif se répète à l'identique à chaque ligne, et tout ce qui n'aura pas ses 63 cycles complets le décalera. C'est notre détecteur de vol.

Un dernier réglage, cosmétique celui-là : nos huit créatures passent **en noir**. Ce n'est pas une panne — c'est le seul moyen de les distinguer encore. Le fond, désormais, change de couleur toutes les six microsecondes et parcourt les seize teintes de la palette ; n'importe quelle couleur de sprite s'y noierait par moments. Le noir, lui, tranche sur les quinze autres. Vous verrez donc huit silhouettes, et c'est voulu.

(Ce listing commence par la mise en place des huit sprites, identique à celle du précédent ; la nouveauté tient dans les quatre dernières lignes, celles du chronomètre.)

```

;-----
; 20 MILLISECONDES – chapitre 6 : les huit voleurs
;
; Le chronomètre visuel du chapitre 4 (inc en boucle), élargi au
; fond de l'écran et calé sur 21 cycles – avec les huit sprites
; posés en travers du milieu de l'image. Regardez les colonnes
; de couleur à hauteur des créatures.
;
; Assembler :   acme voleurs.a
; Lancer :      LOAD"VOLEURS",8,1 puis RUN
;               (RUN/STOP + RESTORE pour sortir)
;-----
!to "voleurs.prg", cbm

* = $0801                ; l'amorce du chapitre 1 : 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                    ; silence : plus personne ne nous interrompt

    lda #dessin / 64       ; les huit pointeurs sur le même dessin
    sta $07f8
    sta $07f9
    sta $07fa
    sta $07fb
    sta $07fc
    sta $07fd
    sta $07fe
    sta $07ff

    lda #140               ; même Y : les huit sur la même rangée
    sta $d001
    sta $d003
    sta $d005
    sta $d007
    sta $d009
    sta $d00b
    sta $d00d
    sta $d00f

    lda #60                ; les X, un par un
    sta $d000
    lda #92
    sta $d002
    lda #124
    sta $d004
    lda #156
    sta $d006
    lda #188
    sta $d008
    lda #220
    sta $d00a
    lda #252
    sta $d00c
    lda #28                ; le huitième : 284 = 256 + 28
    sta $d00e
    lda #%10000000
    sta $d010

```

(Les huit voleurs : voici la **suite du même fichier**, à coller directement après la première partie. Les deux moitiés ne font qu'un seul programme.)

```

lda #0                ; les huit en NOIR : seule couleur qui tranche
sta $d027              ; sur un fond qui parcourt les 16 teintes
sta $d028
sta $d029
sta $d02a
sta $d02b
sta $d02c
sta $d02d
sta $d02e

lda #%11111111        ; les huit allumés
sta $d015

; --- le chronomètre : 6 + 6 + 6 + 3 = 21 cycles par tour,
; et 21 x 3 = 63 = la ligne entière. Motif bien vertical.
boucle inc $d020        ; 6 cycles – la bordure change de couleur
inc $d021              ; 6 cycles – le fond aussi
inc $d020              ; 6 cycles – la bordure encore
jmp boucle             ; 3 cycles – et on recommence

;-----
; Le dessin : 21 lignes de 3 octets = 63 octets.
; Adresse OBLIGATOIREMENT multiple de 64 ($0a00 = 40 x 64).
; Un « 1 » = un pixel de la couleur du sprite, un « 0 » = rien
; du tout : on voit l'écran à travers.
;-----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

Ce qu'on observe :



L'écran entier est devenu un chronomètre : de fines rayures horizontales (la couleur avance un peu à chaque ligne) découpées par de **grandes frontières verticales**, chacune étant un inc exécuté. Trois choses à y lire.

En haut, les frontières sont parfaitement verticales. Dans la bordure supérieure, le VIC travaille toujours — il lit à vide, il va même chercher les pointeurs des huit sprites — mais tout cela tient dans sa propre moitié de microseconde : il ne **réquisitionne** rien. Le processeur garde ses 63 cycles, et la boucle de 21 cycles retombe au même endroit à chaque ligne, au pixel près.

Dans la zone d'affichage, les frontières montent en petites marches. Un cran de huit pixels — un cycle — **toutes les huit lignes** : c'est la Bad Line du chapitre 4, imperturbable. Pourquoi un si petit cran, alors qu'une Bad Line vole une quarantaine de cycles ? Parce que notre chronomètre ne mesure pas le vol : il mesure ce qui **dépasse** un nombre entier de tours, et une aiguille unique ne compte pas les tours. Ce reste se calcule, et il permet de resserrer la mesure — à condition d'être honnête sur ce qu'il prouve. Un cran d'un cycle nous dit seulement que les cycles laissés au processeur tombent à **un cycle près d'un multiple de 21** : ce peut être 20, 22, 41, 43... Le vol correspondant vaudrait donc 43, 41, 22 ou 20 cycles. Croisons avec ce que dit la documentation — une Bad Line prend entre 40 et 43 cycles — et il n'en reste que **deux** : 41 ou 43.

Nous ne saurons pas lequel, et c'est très bien ainsi : la leçon est justement là. Une mesure seule laissait quatre possibilités, une fourchette documentée seule en laissait quatre aussi (40, 41, 42, 43), et les deux ensemble n'en laissent que deux. On a gagné en précision sans faire dire à personne plus qu'il ne sait — et pour trancher entre 41 et 43, il faudrait un instrument plus fin qu'une capture d'écran.

Retenez surtout ce que cette fourchette signifie : **le vol n'est pas un nombre fixe**. Il dépend de l'endroit où le processeur en était dans son instruction quand le VIC a levé la main. Une même Bad Line ne coûte pas tout à fait la même chose d'une ligne à l'autre — et au chapitre 9, cette petite irrégularité décidera du sort de trois programmes.

Et sur la bande des créatures, tout le motif est poussé de côté d'un bloc. Plus de petites marches : un décalage franc, de l'ordre d'une centaine de pixels, sur une vingtaine de lignes — les vingt-et-une du dessin, plus une juste au-dessus. Les sprites, immobiles, muets, sans une instruction à eux, viennent de voler du temps au processeur.

L'explication — ce que coûte vraiment un sprite

Un sprite fait deux sortes d'accès à la mémoire. **Le pointeur** d'abord : une lecture de son casier (\$07f8 pour le sprite 0, \$07f9 pour le sprite 1, et ainsi de suite) à **chaque** ligne de balayage, dans la **première** moitié du cycle — la moitié du VIC. Elle ne coûte donc rien au processeur, et elle a lieu **même si le sprite est éteint** : un sprite éteint est vraiment gratuit. **Les trois octets du dessin** ensuite, uniquement si le sprite est affiché sur cette ligne : ils occupent les **trois demi-cycles qui suivent immédiatement** la lecture du pointeur, c'est-à-dire la seconde moitié du cycle en cours puis les deux moitiés du cycle suivant. La première et la troisième de ces lectures tombent dans la **moitié du processeur**. Il les perd.

D'où le chiffre à retenir : **environ deux cycles perdus par sprite affiché, par ligne de balayage** — huit sprites sur la même ligne, seize cycles. Plus un supplément, parce que le VIC est un artisan poli : quand il a besoin de la moitié du 6510, il **prévient trois cycles à l'avance** (le signal BA), et pendant ces trois cycles le processeur peut encore terminer des écritures mais pas *commencer* une nouvelle instruction. Le compte théorique est donc $16 + 3 = 19$ **cycles volés**. En pratique moins, car le processeur grappille une partie de la fenêtre d'annonce : les tables de référence donnent **46 à 49 cycles disponibles** sur les 63 d'une ligne portant huit sprites, soit **14 à 17 cycles volés**. Notez la fourchette : les sources primaires ne concordent pas au cycle près, et elles le disent. Sur ce genre de nombre, l'honnêteté est un intervalle, pas une valeur.

Et maintenant, vérifiez. Le faisceau parcourt **8 pixels par cycle** : 14 à 17 cycles volés font 112 à 136 pixels de décalage, l'ordre de grandeur exact du décrochement que vous avez sous les yeux. Vous venez de lire un vol de cycles directement à l'écran, avec une règle graduée et rien d'autre. Voici enfin le pire cas de la machine, celui que tout programmeur de démo connaît par cœur : **une Bad Line qui porte aussi les huit sprites ne laisse que 4 à 7 cycles au processeur**. Sur soixante-trois. De quoi caser deux instructions courtes, pas plus : c'est la limite absolue du Commodore 64, et c'est contre elle que se cassent les effets ambitieux.

Deux précisions sur la forme exacte de ce que vous voyez. **Vingt-deux lignes et non vingt-et-une**, parce que le VIC lit en avance de phase : les données des sprites 3 à 7 sont lues aux cycles 1 à 9 de la ligne où ils s'affichent (l'annexe, à la fin du livre, donne la carte complète des 63 cycles d'une ligne), mais celles des sprites 0, 1 et 2 aux cycles 58, 60 et 62 — **dans la ligne précédente**. Et si tout le motif de la ligne est décalé, pas seulement le morceau situé derrière les créatures, c'est que ces cycles-là tombent hors de l'image (le

retour de ligne à gauche, l'extrême bord droit) : le processeur prend son retard **avant** d'entrer dans la partie visible et le garde jusqu'au bout. On ne voit pas le vol, on voit son effet.

Dernière curiosité, contre-intuitive et vérifiée : **éteindre un sprite « au milieu » ne rend aucun cycle**. Éteignez le sprite 4 alors que les huit sont actifs (`lda $d015 / and #%11101111 / sta $d015`) : le motif ne bouge pas d'un pixel, car pendant le créneau qu'aurait occupé le sprite 4, le VIC annonce déjà qu'il veut le bus pour le sprite 5. Les fenêtres sont jointives ; pour récupérer du temps, il faut libérer une **plage** de créneaux voisins, pas un sprite au hasard. Huit objets libres, donc — mais dans un Commodore 64, le temps appartient à la puce vidéo, et les sprites sont l'une des manières les plus élégantes de le lui donner.

Sous le capot — le cycle de vie complet d'un sprite (le test de sa coordonnée Y, l'allumage de son DMA, ses compteurs internes, son registre à décalage) et les règles de priorité entre sprites sont dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §10. Le décompte du vol de cycles — les deux lectures perdues sur trois, le préavis de trois cycles, et la table des budgets par type de ligne (63 / 46-49 / 23 / 4-7 cycles) — occupe tout son chapitre 2, d'après Pasi Ojala (1992) et les chronogrammes mesurés de Marko Mäkelä (1994). Le coût des instructions de notre chronomètre : chapitre 3, §2.

Au prochain chapitre

Vous savez maintenant fabriquer du retard — vingt-deux lignes, avec huit sprites qui ne font rien. Au chapitre 7, nous ferons l'inverse : **fabriquer du temps**. Vous avez désormais les treize instructions du livre au complet ; ce qui vient ne demande plus d'outil nouveau, seulement de connaître deux compteurs cachés du VIC — que personne ne peut lire, mais que tout le monde peut tromper. À la fin du chapitre, l'écran tombera.

Chapitre 7 — Les compteurs cachés, et l'écran qui tombe

La question

Nous entrons dans la troisième partie du livre, celle où l'on cesse d'observer la machine pour commencer à la détourner. Et la première question à lui poser est celle-ci : nous savons faire apparaître un caractère en écrivant dans un casier. Peut-on faire tomber l'écran **tout entier** sans déplacer un seul octet ?

La réponse est oui, elle tient en une boucle de six instructions, et elle repose entièrement sur la Bad Line du chapitre 4.

Le signet du VIC

Il faut d'abord comprendre une chose que nous avons soigneusement contournée jusqu'ici : le VIC ne sait pas « où il en est » dans l'écran de la façon dont vous et moi l'imaginerions.

Il tient deux compteurs internes, invisibles depuis le programme, et c'est ce couple qui fait tout le travail. Le premier retient **quelle ligne de texte** il est en train d'afficher — appelons-le son signet. Le second retient **quelle ligne de pixels**, de zéro à sept, à l'intérieur de cette ligne de texte.

Voici le point crucial, et il découle directement du chapitre 4 : **sans Bad Line, aucune nouvelle rangée de texte n'est lue**. La Bad Line n'est pas seulement le moment où le VIC vole des cycles : c'est son *signal de départ* pour une rangée. Pas de Bad Line, pas de lecture — le VIC reste sur place.

Une nuance à garder en réserve, car le chapitre 9 en fera son affaire : la Bad Line déclenche la **lecture** d'une rangée, mais le signet, lui, ne se déplace d'une rangée à la suivante qu'une fois les huit lignes de pixels épuisées. Bad Line et avancement du signet sont deux événements distincts ; ici ils vont ensemble, et c'est tout ce dont nous avons besoin.

Or nous savons exactement ce qui déclenche une Bad Line : l'égalité entre les trois derniers bits du numéro de ligne et le cran de défilement vertical, ce réglage à huit positions rangé dans `$d011`. Un registre que nous pouvons écrire, ligne après ligne, aussi souvent que nous le voulons.

L'idée du trucage se formule alors d'elle-même : **si l'égalité ne se produit jamais, le VIC ne lit plus rien de nouveau**. Il attend, l'affichage reste suspendu, et quand on lui rend enfin sa liberté, il reprend sa lecture là où il l'avait laissée — quarante lignes plus bas. Vu de l'écran, tout le contenu est tombé.

Ce trucage a un nom, donné par les programmeurs de démos des années 1980 : **FLD**, pour *Flexible Line Distance* — « distance flexible entre les lignes ». C'est le plus simple des grands trucages du Commodore 64, et une bonne porte d'entrée avant le chapitre 9.

Le piège du décalage, et pourquoi il faut viser deux lignes en avance

Notre boucle va lire le numéro de ligne courant et écrire un cran interdit. Question : quel cran ?

Le réflexe serait d'écrire un cran différent de la ligne courante — disons $(\text{ligne} + 1) \& 7$. C'est faux, et c'est une erreur instructive. Réfléchissez à ce qui se passe pendant que la ligne i est en cours : notre boucle y écrit le cran $(i+1) \& 7$. Puis la ligne $i+1$ commence — et le test de la Bad Line, lui, a lieu **au tout début de la ligne**, avant que notre boucle ait eu le temps de tourner à nouveau. La valeur encore en place est $(i+1) \& 7$, et le numéro de ligne est maintenant $i+1$. Égalité. Bad Line. Trucage raté.

Il faut donc viser **deux lignes en avance** : écrire $(i+2) \& 7$ pendant la ligne i . Cette valeur n'est égale ni à $i \& 7$ (pas de Bad Line tout de suite), ni à $(i+1) \& 7$ (pas de Bad Line au début de la ligne suivante, même si notre boucle est en retard). Le « +2 » n'est pas une superstition : c'est exactement la marge dont nous avons besoin pour que le retard de notre boucle ne nous coûte rien.

Retenez le raisonnement plus que la formule. Programmer le VIC, c'est constamment se demander : *que vaut ce registre à l'instant où la machine le consulte ?* — et non pas à l'instant où nous l'écrivons.

L'expérience

```
!to "chute.prg", cbm

DEPART = $32          ; ligne ou commence la chute (50)
FIN     = $5a          ; ligne ou on rend la main (90) -> 40 lignes

* = $0801             ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence

; --- attendre le haut de l'ecran ---
trame   lda $d012
        cmp #DEPART
        bne trame

; --- pendant N lignes : deplacer le cran pour que l'egalite n'arrive JAMAIS ---
chute   ldx $d012        ; 4 : la ligne en cours
        lda cran,x       ; 4 : un cran interdit pour les DEUX lignes qui viennent
        sta $d011        ; 4 : plus de Bad Line
        lda $d012        ; 4 : sommes-nous au bout ?
        cmp #FIN         ; 2
        bne chute       ; 3 -> 21 cycles par tour, 3 tours par ligne

; --- on rend la main : les Bad Lines reprennent, l'ecran reprend sa lecture ---
        lda #$1b         ; cran 3, ecran allume, 25 lignes
        sta $d011
        jmp trame        ; et on recommence a la trame suivante

; --- la table des crans « interdits » ---
!align 255, 0
cran
!for i, 0, 255 {
    !byte $18 | ((i + 2) & 7) ; $18 = ecran allume + 25 lignes
}
```

Trois notations de l'assembleur apparaissent ici pour la première fois, et elles ne coûtent rien à comprendre. `DEPART = $32` donne un **nom** à un nombre : l'assembleur remplacera `DEPART` par `$32` partout, et vous pourrez changer la valeur en un seul endroit. `!for i, 0, 255 { ... }` est une boucle **de fabrication** : elle ne tourne pas dans la machine, elle tourne dans l'assembleur, et son résultat est une table de 256 octets écrite dans le programme. Enfin `&` et `|` sont les mêmes opérations que les instructions `and` et `ora` du chapitre 6, mais calculées à la fabrication.

Et surtout, la clé de lecture de cette table : **`(i + 2) & 7` garde les trois derniers bits de `i + 2`** — exactement ces trois bits dont le chapitre 4 disait qu'ils déclenchent la Bad Line quand ils tombent sur le cran de défilement. Le `| $18` rallume par-dessus les interrupteurs qu'on veut conserver. Cette petite ligne *est* le truage.

Reconnaissez le motif signature du chapitre 3 — `ldx $d012` suivi de `lda cran,x` — mais mis au service d'autre chose que de la couleur. Toute la ruse a été calculée par l'assembleur, d'avance, dans la table `cran` : à l'affichage, il ne reste qu'à lire et écrire. La boucle coûte 21 cycles, elle tourne donc trois fois par ligne, et cette générosité est notre sécurité : même si l'une des trois passes tombe au mauvais moment, les deux autres tiennent le registre.

Notez enfin le `$18` de la table : il rallume l'écran et garde la fenêtre à 25 lignes à chaque écriture. Un registre matériel se réécrit **en entier** — on ne peut pas y toucher un seul interrupteur sans dire aussi ce qu'on veut pour les autres. Nous saurions n'en changer qu'un, avec le `and/ora` du chapitre 6 ; ici c'est inutile, puisque la table connaît d'avance la valeur complète à écrire.

Ce qu'on observe :



Capture réelle. Le texte est intact, à sa place dans la mémoire — c'est sa lecture qui a été suspendue quarante lignes durant.

Mesuré dans la capture : la première rangée de texte, qui commence normalement à la ligne 43, commence ici à la ligne 83. **Quarante lignes exactement** — la différence entre nos constantes FIN et DEPART. Changez FIN, la chute change d'autant.

Et le vide au-dessus ? Ce n'est pas du noir, ni un rideau : c'est le VIC qui, privé de nouvelle ligne de texte, entre dans ce que la documentation appelle son **état de repos**. Il continue de lire consciencieusement une adresse, toujours la même, et traite ce qu'il y trouve comme si tous les pixels étaient éteints. D'où la couleur de fond, uniforme, proprement affichée. Le VIC n'a pas « rien fait » : il a affiché du vide avec application.

Ce que vous venez d'acquérir

Prenez la mesure du changement. Jusqu'au chapitre 6, nous écrivions dans des casiers pour dire à la machine *quoi* afficher. Ici, pour la première fois, nous avons écrit dans un casier pour lui dire *quand* — et nous avons obtenu un effet que son matériel ne propose nulle part : un défilement vertical de tout l'écran, à coût presque nul, sans déplacer un octet de la mémoire.

C'est la définition même des trucages du C64. On ne trouve pas une fonction cachée : on observe une règle (« le signet n'avance qu'à la Bad Line »), on remarque qu'un de ses ingrédients est sous notre contrôle, et on s'en sert à contretemps.

Sous le capot — le mécanisme exact (les compteurs VC et RC, la remise à zéro du compteur de ligne, la transition entre l'état d'affichage et l'état de repos, et le catalogue complet des effets qui en découlent : FLD, Linecrunch, VSP) est spécifié dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1 : §8 pour les compteurs VC et RC, §7 pour l'état de repos, et §16.2 pour le FLD lui-même.

Au prochain chapitre

Nous avons fait tomber l'écran en empêchant un événement de se produire. Au chapitre suivant, nous ferons disparaître la **bordure** en empêchant deux comparaisons de tomber juste — et un sprite se promènera là où l'écran n'existe pas.

Chapitre 8 — Ouvrir la bordure

La question

Il y a un cadre autour de l'image du Commodore 64. Une trentaine de lignes en haut, autant en bas, une bande de chaque côté — une zone où, dit-on, « on ne peut rien afficher ».

Et pourtant les démos des années 1980 y font promener des sprites. Comment ?

À partir de ce chapitre, je vous dois un avertissement rassurant : **vous n'apprenez plus aucune instruction nouvelle**. Les treize que vous connaissez suffiront jusqu'à la dernière page, FLI compris. Tout ce qui vient désormais n'est pas une question d'outillage, mais de compréhension du matériel — et vous en savez déjà assez.

La bordure n'est pas une zone : c'est un robinet

Voilà l'idée qu'il faut désapprendre. Nous imaginons naturellement la bordure comme une région de l'écran, quelque chose de géographique : « ici c'est la fenêtre, là c'est le cadre ». Le VIC ne fonctionne pas comme ça du tout.

Il tient un **interrupteur** — les électroniciens disent une bascule — qui répond à une seule question : « en ce moment, est-ce que je sors la couleur de bordure ? ». Quand cet interrupteur est à 1, le VIC recouvre tout : graphique, sprites, absolument tout. Quand il est à 0, l'image passe.

Et cet interrupteur est manœuvré par des **comparaisons**. Deux nous intéressent :

- quand le numéro de ligne atteint la comparaison **du bas** — 251 en mode 25 rangées de texte — la bascule est **mise à 1**, et la bordure du bas commence ;
- quand il atteint la comparaison **du haut** — 51 dans le même mode — la bascule est **remise à 0**, et la fenêtre d'affichage commence.

Notez maintenant la phrase de la documentation qui contient tout le trucage, et lisez-la deux fois : *la comparaison n'est vraie que si la valeur est atteinte exactement — ce n'est pas un test d'intervalle*.

Le VIC ne se demande pas « suis-je en dessous de la ligne 251 ? ». Il se demande « suis-je à la ligne 251 ? ». Et si la réponse est non, il ne se passe rien. Jamais. Il n'y a aucune session de rattrapage.

Deux valeurs, un seul interrupteur

Reste à savoir comment esquiver une comparaison. Nous ne pouvons pas empêcher le faisceau d'atteindre la ligne 251 — mais nous pouvons changer **la valeur à laquelle il compare**.

Le C64 propose en effet deux hauteurs de fenêtre : 25 rangées de texte (le réglage habituel) ou 24 rangées, un cran plus serré. Ce choix tient dans un seul interrupteur du casier \$d011, et il change les deux valeurs comparées :

Mode	Comparaison du haut	Comparaison du bas
25 rangées	ligne 51	ligne 251
24 rangées	ligne 55	ligne 247

Le plan s'écrit alors tout seul. Restons en 25 rangées quand la ligne 247 passe : la valeur comparée est 251, l'égalité est fausse, rien ne se produit. Puis, avant que la ligne 251 n'arrive, basculons en 24 rangées : la valeur comparée devient 247... mais cette ligne-là est déjà derrière nous. L'égalité est fausse une seconde fois.

Deux comparaisons esquivées, et la bascule n'est jamais mise à 1. **La bordure du bas n'aura pas lieu**, non pas parce qu'on l'a effacée, mais parce que personne n'a donné l'ordre de l'allumer.

Changer un interrupteur sans toucher aux autres

Un détail de méthode, promis au chapitre 7. Le casier \$d011 contient huit interrupteurs qui n'ont rien à voir entre eux : l'écran allumé, la hauteur de la fenêtre, le cran de défilement... Or on ne peut pas en écrire un seul : toute écriture remplace les huit.

C'est là que servent les deux instructions du chapitre 6 :

```
lda $d011
and #%11110111      ; ce zero-la ETEINT l'interrupteur, les sept autres intacts
sta $d011
```

Lire, modifier un bit, réécrire. and avec un zéro éteint ; ora avec un un allume. C'est le geste le plus courant de toute la programmation du C64, et vous venez de le voir dans son usage le plus naturel : **respecter ce que le voisin a réglé.**

L'expérience

```

!to "sansbord.prg", cbm

* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
; --- une creature, posee SOUS le bas de l'ecran ---
lda #40                  ; dessin au bloc 40 ($0a00)
sta $07f8
lda #160
sta $d000                ; X
lda $d010
and #%11111110          ; X ne depasse pas 255
sta $d010
lda #250                 ; Y = 250 : dans la « bordure » du bas
sta $d001
lda #7                   ; jaune
sta $d027
lda $d015
ora #%00000001          ; sprite 0 allume
sta $d015

lda #0                   ; l'octet fantome : voir la fin du chapitre
sta $3fff

sei                      ; silence

; --- a chaque trame : empecher les DEUX comparaisons de tomber juste ---
attend1 lda $d012
cmp #250                 ; la ligne 251 va arriver...
bne attend1
lda $d011
and #%11110111          ; ...on passe en 24 lignes : la valeur
sta $d011                ; comparee devient 247, deja passee

attend2 lda $d012
cmp #252                 ; la ligne 251 est passee sans rien declencher
bne attend2
lda $d011
ora #%00001000          ; on remet 25 lignes pour la trame suivante
sta $d011

jmp attend1

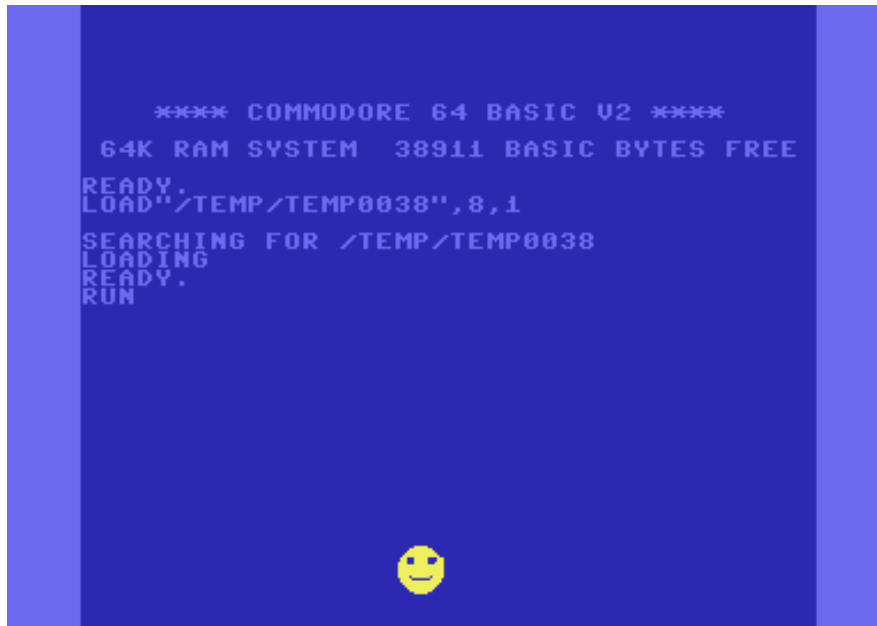
; -----
; Le dessin : 21 lignes de 3 octets = 63 octets.
; Adresse OBLIGATOIREMENT multiple de 64 ($0a00 = 40 x 64).
; Un « 1 » = un pixel de la couleur du sprite, un « 0 » = rien
; du tout : on voit l'écran à travers.
; -----
* = $0a00
dessin
!byte %00000000,%01111110,%00000000
!byte %00000001,%11111111,%10000000
!byte %00000011,%11111111,%11000000
!byte %00000111,%11111111,%11100000
!byte %00001111,%11111111,%11110000
!byte %00011100,%01111110,%00011100
!byte %00011100,%01111110,%00011100
!byte %00111100,%01111110,%00111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111111,%11111111,%11111100
!byte %00111110,%11111111,%01111100
!byte %00011111,%00000000,%11111000
!byte %00011111,%11111111,%11111000
!byte %00001111,%11111111,%11110000
!byte %00000111,%11111111,%11100000
!byte %00000011,%11111111,%11000000
!byte %00000001,%11111111,%10000000
!byte %00000000,%01111110,%00000000

```

Le dessin du sprite est celui du chapitre 6, réimprimé à la fin du listing pour que vous puissiez le taper — ou le coller — d'un seul tenant. Notez au passage sa ligne `* = $0a00` : elle n'est pas décorative. Sans

elle, les 63 octets se rangeraient juste après le code, et le sprite irait chercher son dessin là où il n'y a rien. Remarquez la position : $Y = 250$, c'est-à-dire *sous* la limite basse de l'écran. Dans une machine qui respecte ses propres règles, cette créature est invisible.

Ce qu'on observe :



Capture réelle. La créature est là où l'écran n'existe pas.

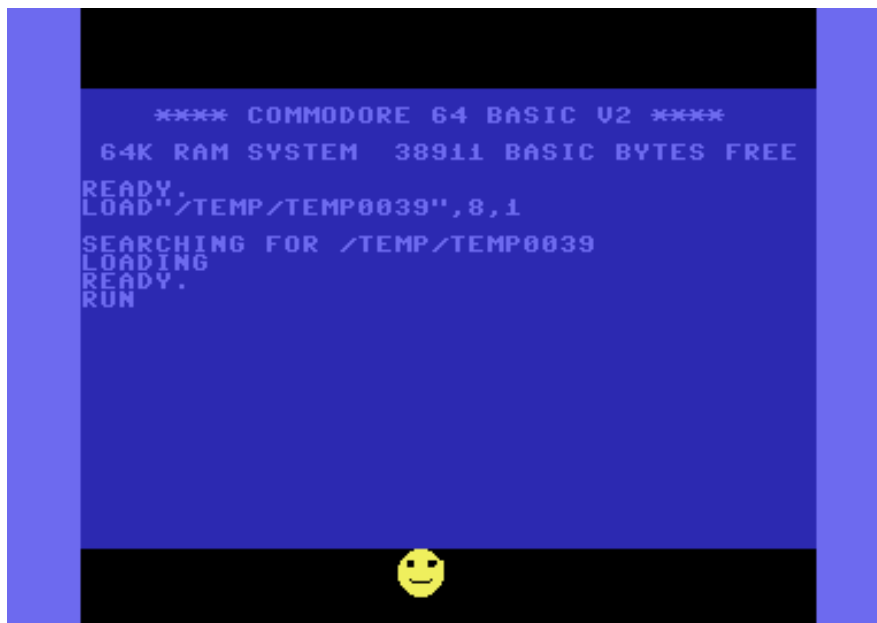
Et une surprise, que la mesure confirme : **il n'y a plus aucune bordure horizontale**, ni en bas ni en haut. J'ai relevé la colonne centrale de la capture — 272 lignes, une seule couleur, pas un pixel de cadre — alors qu'une capture normale montre 35 lignes de bordure en haut et 37 en bas.

Pourquoi le haut aussi ? Relisez les deux règles. La comparaison basse **met** la bascule à 1 ; la comparaison haute la **remet** à 0. Nous avons empêché la mise à 1 ; il ne reste donc rien à remettre à 0 en haut de l'image suivante. L'interrupteur est resté à 0 d'un bout à l'autre. Nous voulions ouvrir une porte, nous avons ouvert le couloir entier — et c'est logique, pas magique.

L'octet fantôme

Une dernière curiosité, et elle vaut le détour. Retirez la ligne `lda #0 / sta $3fff` de notre programme — ou mieux, remplacez le `#0` par `#$ff` pour forcer le phénomène.

Ce qu'on observe :



Capture réelle, avec \$3fff mis à \$ff exprès. Les zones que nous venons d'ouvrir sont noires : le VIC y affiche quelque chose, et ce quelque chose vient d'un seul casier.

Souvenez-vous de l'état de repos du chapitre 7 : quand le VIC n'a pas de ligne de texte à afficher, il ne s'arrête pas pour autant — il continue de lire, toujours à la même adresse, \$3fff. Et il traite ce qu'il y trouve comme des pixels : chaque bit à 1 devient un pixel noir, chaque bit à 0 laisse voir la couleur de fond.

Dans les zones que nous venons d'ouvrir, nous ne regardons donc pas « rien ». Nous regardons le contenu d'un unique casier de la mémoire, répété à l'infini, huit pixels par huit pixels. Écrivez zéro dedans, et le vide redevient propre. Les programmeurs de démos le savent depuis toujours : la première ligne de tout code qui ouvre une bordure est presque toujours celle qui nettoie \$3fff.

Sous le capot — les deux bascules de bordure (principale et verticale), leurs quatre comparateurs, les six règles exactes qui les manœuvrent et les valeurs pour chaque combinaison de mode sont spécifiés dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 1, §11 (« Unité de bordure »). L'état de repos et le rôle de \$3fff sont au §7 du même chapitre.

Au prochain chapitre

Nous avons esquivé une comparaison. Nous avons empêché une Bad Line. Il reste à faire l'inverse : en **provoquer** une, à chaque ligne, pour obliger le VIC à relire ses couleurs 200 fois par image au lieu de 25. C'est le trucage le plus célèbre de la machine, et le dernier de ce livre.

Chapitre 9 — Le grand final : toutes les couleurs à la fois

La question

Le Commodore 64 a une réputation tenace : ses couleurs sont grossières. Dans son mode graphique le plus simple, l'écran est découpé en cellules de 8×8 pixels, et **chaque cellule ne peut montrer que deux couleurs**. Deux couleurs pour 64 pixels : c'est ce qui donne aux images de cette machine leur aspect de mosaïque. (Il existe un mode multicolore qui en autorise quatre, mais au prix de la moitié de la finesse horizontale : le compromis déplace le problème, il ne le résout pas.)

Et pourtant. Ouvrez n'importe quelle galerie de graphismes C64 et vous trouverez des portraits aux dégradés impossibles, des ciels dégradés ligne par ligne. Comment ?

La réponse est le trucage le plus célèbre de la machine, et vous avez déjà tout ce qu'il faut pour le comprendre : il ne consiste qu'à **provoquer** ce que le chapitre 7 s'employait à empêcher.

Ce qu'est vraiment le mode graphique

Une mise au point d'abord, car elle est contre-intuitive. En mode graphique, la matrice vidéo du chapitre 5 — ces 1000 casiers qui contenaient un caractère chacun — ne contient plus de caractères. Elle contient **des couleurs** : dans chaque octet, les quatre bits hauts donnent la couleur des pixels allumés de la cellule, les quatre bits bas celle des pixels éteints. Les pixels eux-mêmes, les 8000 octets qui disent quel point est allumé, vivent ailleurs.

Notez la conséquence : les couleurs ne sont pas lues avec les pixels. Elles sont lues avec la matrice — c'est-à-dire **une seule fois toutes les huit lignes**, pendant la Bad Line. Voilà l'origine exacte de la limitation : ce n'est pas que le VIC refuse plus de deux couleurs par cellule, c'est qu'il ne *demande* leur couleur qu'une fois par ligne de texte.

Vous voyez déjà où nous allons.

L'idée : une Bad Line par ligne

Si le VIC relit ses couleurs à chaque Bad Line, et si nous savons manœuvrer le cran de défilement qui les déclenche — nous l'avons fait au chapitre 7, mais pour les **empêcher** — alors retournons le geste et forçons en une **sur chaque ligne raster**. Le VIC relira ses couleurs 200 fois par image au lieu de 25. Chaque bande de 8 pixels de large et 1 pixel de haut pourra avoir ses propres couleurs.

C'est le **FLI**, pour *Flexible Line Interpretation*, inventé par les programmeurs de démos à la fin des années 1980. Il coûte cher — nous verrons combien — mais il change complètement ce que la machine peut montrer.

Deux obstacles se dressent, et la documentation les nomme tous les deux.

Premier obstacle : le VIC relit toujours les mêmes adresses. Quand une Bad Line survient avant la fin de la ligne de texte en cours, le VIC ne fait pas avancer son signet — il relit donc exactement les 40 mêmes cellules. Nous aurions des couleurs relues 200 fois, mais identiques ! La parade est brutale et élégante : nous ne changeons pas les données, nous changeons **l'endroit où le VIC va les chercher**. Souvenez-vous du chapitre 5 : §018 dit où est la matrice. Nous préparons donc **huit matrices** en mémoire, et nous en changeons à chaque ligne. Huit suffisent, parce que la ligne raster L lit la rangée $L/8$ de la matrice numéro $L \bmod 8$ — les huit lignes d'une même rangée de cellules puisent dans huit matrices différentes.

Second obstacle : il y a un instant précis pour agir. L'écriture qui crée la Bad Line ne doit pas arriver avant le **cycle 14** de la ligne. Plus tôt, et le VIC remet à zéro son compteur de ligne de pixels : il réafficherait indéfiniment la même rangée de pixels. Vous verrez cet accident de vos yeux dans un instant — je m'y suis cogné deux fois.

Trois tentatives, et ce que chacune enseigne

Je pourrais vous donner le programme qui marche. Il vaut mieux que je vous raconte comment je l'ai trouvé : les deux échecs sont plus instructifs que la réussite.

Première tentative. Une boucle qui compte : 200 lignes à peindre, un compteur qui descend, et deux tables lues à l'index du compteur. Vingt et un cycles par tour. Résultat : les trente-deux premières lignes sont magnifiques, puis l'écran se change en bouillie répétitive — la signature exacte du compteur de pixels remis à zéro. La boucle avait dérivé.

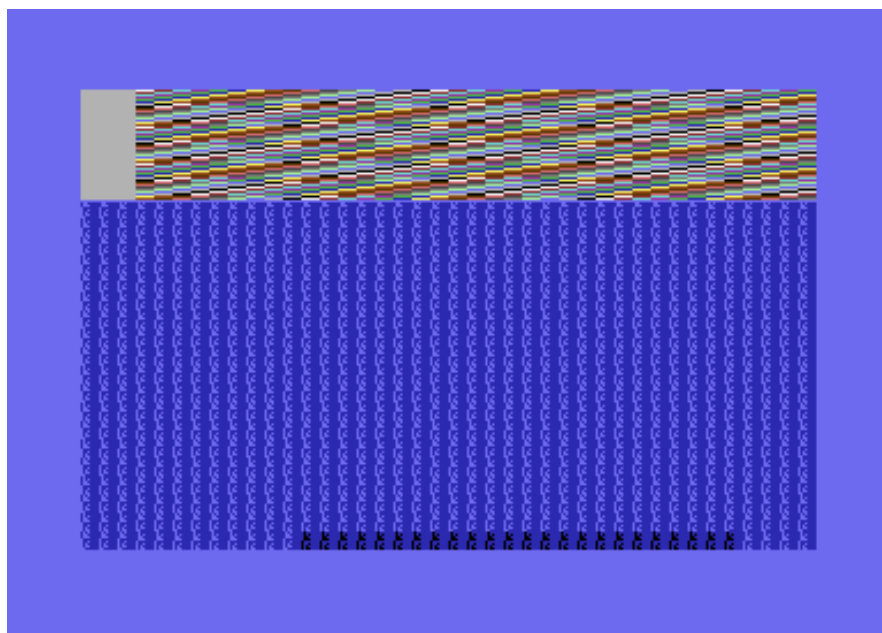
Deuxième tentative. Calculons mieux. Une ligne dure 63 cycles ; sur une Bad Line, la table de référence donne 40 cycles réquisitionnés, donc 23 laissés au processeur. Vous vous souvenez peut-être du chapitre 6, où la mesure donnait plutôt 41 ou 43 : les deux sont vraies, parce que ce vol **n'est pas constant**. Retenez cette phrase, tout ce chapitre en découle. Prenons pour l'instant les 23 cycles de la table. Notre boucle en consommait 21 : deux cycles d'avance à chaque ligne, et l'écriture finissait par tomber avant le fameux cycle 14. Il faut donc *ralentir* la boucle de deux cycles — c'est le rôle du nop, l'instruction qui ne fait rien pendant deux cycles, la cale d'épaisseur des programmeurs du C64. C'est la quatorzième instruction du 6510 que ce livre prononce, et la seule qu'il n'enseigne pas : vous ne la verrez que dans ce programme-ci, celui qui ne marche pas. Vingt-trois cycles pile. Résultat : le même effondrement, seulement repoussé — quarante-huit lignes justes au lieu de trente-deux. J'avais acheté seize lignes, pas une solution.

Ce deuxième échec est le plus intéressant du livre, parce qu'il révèle une **erreur de raisonnement**, pas une erreur de calcul. Mon compteur supposait « un tour de boucle = une ligne d'écran ». Cette hypothèse est *invérifiable de l'intérieur* : rien, dans mon programme, ne demandait à la machine où en était réellement le faisceau. Le compteur et la réalité racontaient deux histoires, et rien ne les rapprochait.

Voici la boucle fautive — comptez-la, elle fait bien ses 23 cycles :

```
sync    lda $d012
        cmp #$33          ; on entre dans la fenetre d'affichage
        bne sync
        ldx #200          ; 200 lignes a peindre... croit-on
rate    lda t11x,x         ; 4 : « le cran de la ligne que je CROIS peindre »
        sta $d011         ; 4
        lda t18x,x         ; 4
        sta $d018         ; 4
        nop               ; 2 : la cale d'epaisseur
        dex               ; 2
        bne rate          ; 3 -> 23 cycles
```

Ces deux tables t11x et t18x sont celles de la version finale, décalées d'un cran ; le programme complet est dans le fichier flirate.a fourni, si vous voulez reproduire l'échec. Et voici ce qu'il donne à l'écran — car il faut le voir pour comprendre la suite :



Capture réelle de la deuxième tentative. Les 48 premières lignes sont justes, puis le motif se répète indéfiniment : le VIC réaffiche la même rangée de pixels, parce que notre écriture est passée du bon côté au mauvais côté du cycle 14.

Troisième tentative — celle qui marche. J'ai jeté le compteur. À chaque tour, la boucle demande au VIC où il en est, et lui donne ce que *cette* ligne réclame :

```
fli    ldx $d012          ; 4 : le motif signature du chapitre 3
      lda t11,x           ; 4 : le cran qui FORCE la Bad Line ici
      sta $d011           ; 4
      lda t18,x           ; 4 : la matrice de cette ligne
      sta $d018           ; 4
      jmp fli             ; 3 -> 23 cycles pile
```

Vous connaissez ces six lignes : c'est le motif signature du chapitre 3, `ldx $d012` suivi d'un `lda table,x`, exactement comme la bande-annonce du chapitre 0. Mais son effet est maintenant tout autre : la boucle est **auto-correctrice**. Si un décalage survient, le tour suivant lit la vraie ligne et écrit la vraie valeur. Il n'y a plus de dérive possible, parce qu'il n'y a plus rien à faire dériver — aucun état interne, aucune hypothèse, juste une question posée à la machine, 15 000 fois par seconde.

Et remarquez pourquoi cette forme-là résiste alors que les deux autres ont cédé : puisque le vol de la Bad Line n'est pas constant, **aucune boucle à cadence fixe ne peut suivre le faisceau bien longtemps** — 21 ou 23 cycles, la dérive n'était qu'une question de patience. Une boucle qui redemande la ligne à chaque tour, elle, se moque de l'irrégularité.

Et le plus beau : **le voleur du chapitre 4 est devenu notre métronome**. Chaque Bad Line que nous provoquons gèle le processeur jusqu'au même cycle de la ligne ; il reprend donc toujours au même endroit. Ce gel que nous avons découvert comme une nuisance est ce qui tient l'ensemble en rythme, sans interruption, sans synchronisation savante. La machine nous vole du temps et, ce faisant, nous donne l'heure.

Où le VIC va chercher tout ça

Trois valeurs du listing qui suit méritent d'être déchiffrées, sinon vous les prendriez par la foi — et ce livre s'y refuse.

La banque. Le VIC n'a que quatorze fils d'adresse : il ne voit que 16 kilo-octets à la fois, une **banque**, choisie dans un casier du CIA (`$dd00`) — oui, le portier du chapitre 0, que nous devons laisser tranquille : il détient aussi cette clé-là, et c'est la seule fois que nous le dérangerons. Nous avons besoin de place pour huit matrices et un bitmap ; nous démenageons donc son regard vers la banque `$4000-$7fff`. Un piège attend là : les deux bits qui choisissent la banque sont **câblés à l'envers** — écrire `%10` sélectionne la banque n° 1. Et une conséquence en cascade, qui est une bonne question à se poser : l'octet fantôme du chapitre 8 vivait en `$3fff`, la dernière adresse *vue par le VIC*. Dans la banque 1, cette même adresse vue devient `$7fff` en mémoire réelle. C'est donc là qu'il faut écrire notre zéro — j'avais commencé par me tromper.

\$30 dans \$d011. Le casier aux huit interrupteurs (annexe, page de référence) : ici l'interrupteur du mode **graphique** allumé, l'affichage allumé, la fenêtre réglée sur **24 rangées** — et les trois bits du bas laissés au cran de défilement, que nos tables font varier. Pourquoi 24 et non 25 ? La réponse est à la fin du chapitre, et elle vaut le détour.

<< 4 et | 8 dans \$d018. << est une opération de l'assembleur, pas une instruction : `x << 4` décale les bits de quatre rangs vers la gauche, ce qui range un nombre de 0 à 15 dans les quatre bits du haut de l'octet. C'est exactement le « multiplié par 16 » du chapitre 5, écrit autrement. Or ce sont précisément ces quatre bits qui disent au VIC **où** est la matrice : y écrire le numéro de matrice suffit à en changer. Deux nuances par rapport au chapitre 5, où nous faisons la même chose : ce numéro se compte désormais **depuis le début de la banque** (la matrice n° 1 est en `$4400`, pas en `$0400`), et les trois bits du bas, qui désignaient le générateur de caractères en mode texte, désignent le **bitmap** en mode graphique. Le `| 8` y allume celui qui le place au milieu de la banque, en `$6000`.

L'expérience

```

!to "fli.prg", cbm

* = $0801                ; 10 SYS 2064
!byte $0b,$08,$0a,$00,$9e,$32,$30,$36,$34,$00,$00,$00

* = $0810
    sei                ; silence

    ; --- le VIC regarde la banque 1 ($4000-$7fff) ---
    lda $dd00
    and #%11111100      ; les deux bits de banque sont INVERSES :
    ora #%00000010      ; %10 -> banque 1
    sta $dd00

    lda #0              ; l'octet fantome du chapitre 8... mais en
    sta $7fff           ; banque 1 il vit en $7fff, pas en $3fff !

; --- LA boucle : 23 cycles, indexee par la ligne raster elle-meme ---
fli    ldx $d012
        lda t11,x
        sta $d011
        lda t18,x
        sta $d018
        jmp fli

; --- les deux tables, indexees par le NUMERO DE LIGNE ---
!align 255, 0
t11 !for L, 0, 255 {
    !byte $30 | (L & 7)    ; bitmap + ecran + 24 LIGNES + cran = L&7
}
!align 255, 0
t18 !for L, 0, 255 {
    !byte ((L & 7) << 4) | 8 ; matrice n° L&7, bitmap en $6000
}

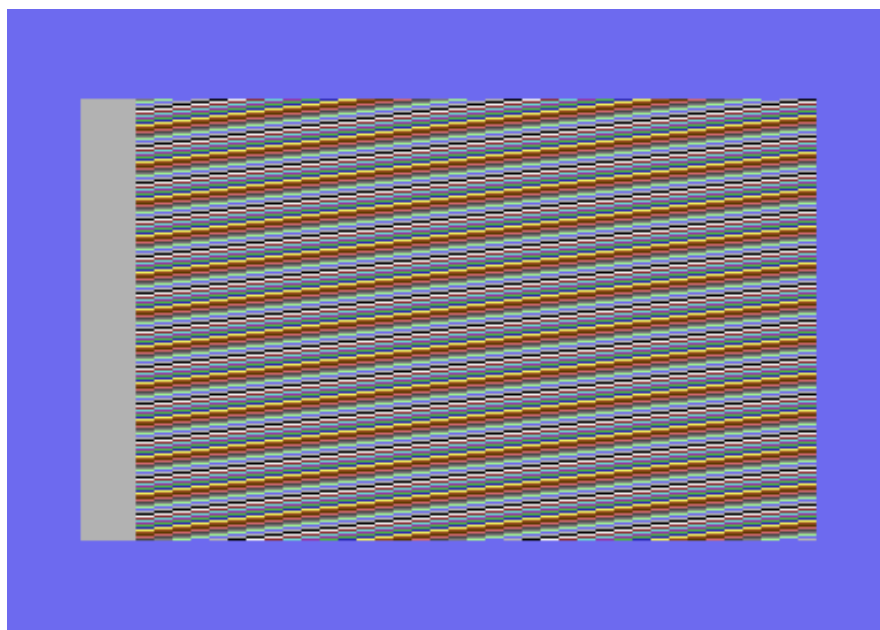
; --- les huit matrices video ($4000, $4400, ... $5c00) ---
!for m, 0, 7 {
    * = $4000 + m * $400
    !for r, 0, 24 {
        !for c, 0, 39 {
            !byte (((r * 8 + m) + c) & 15) << 4
        }
    }
    ; 1000 octets utiles, mais une matrice occupe 1024 : dans les toutes
    ; dernieres lignes de l'image, le compteur du VIC depasse 1000 et lit
    ; ces 24 octets-la. On les colorie donc au lieu de les laisser a zero.
    !for c, 0, 23 {
        !byte (((25 * 8 + m) + c) & 15) << 4
    }
}

; --- le bitmap ($6000) : TOUS les pixels allumes ---
; chaque cellule prend donc la couleur « avant-plan » de sa matrice :
; on ne regarde QUE les couleurs.
* = $6000
!fill 8192, $ff          ; 1024 cellules et non 1000, pour la meme raison

```

Le programme utile fait **treize lignes**. Tout le reste est de la donnée, écrite par l'assembleur avant même que la machine ne démarre : 8 matrices de 1024 octets et 8192 octets de pixels. Le principe du chapitre 3 poussé à sa conclusion — *tout ce qui peut être calculé d'avance ne coûte rien à l'affichage*.

Ce qu'on observe :



Capture réelle. Cent quatre-vingt-douze lignes, chacune avec ses propres couleurs — ce que la fiche technique du Commodore 64 déclare impossible.

J'ai vérifié à la machine plutôt que de vous demander de me croire : **les 192 lignes** de la fenêtre affichent chacune 16 couleurs différentes, et deux lignes voisines n'ont pas les mêmes. Là où le mode graphique normal donnerait 24 rangées de couleurs, nous en avons 192.

La cicatrice

Regardez la bande grise verticale, tout à gauche. Elle n'est pas décorative, et vous ne pouvez pas vous en débarrasser.

Sa cause est d'une précision horlogère. Notre écriture crée la Bad Line au cycle 14 ; le VIC veut sa première cellule au cycle 15. Or, pour que le VIC puisse réellement prendre le bus à un cycle donné, il doit avoir levé la main **trois cycles plus tôt** — c'est un délai câblé dans le silicium, sans dérogation possible. Trois des quarante cellules arrivent donc trop tard : le VIC, portes fermées, lit \$ff au lieu de vos couleurs. Trois cellules, huit pixels chacune : **24 pixels**. J'ai mesuré la bande dans la capture — elle fait 24 pixels exactement.

La documentation de référence, après avoir expliqué ce mécanisme, conclut par une phrase laconique : *there is no way around that*. Il n'y a pas de contournement. Toutes les images FLI de l'histoire du Commodore 64 portent cette cicatrice ; les graphistes ont pris l'habitude de la cacher sous une colonne noire, ou de composer avec elle. C'est, littéralement, la marque de fabrique du trucage.

Pourquoi 24 rangées, et non 25

Reste à honorer la promesse laissée en chemin. Notre programme règle la fenêtre sur 24 rangées, alors que le C64 en propose 25 : nous nous privons de huit lignes d'image. Pourquoi ?

Parce que le trucage a une **frontière**, et qu'elle est écrite noir sur blanc dans la définition de la Bad Line : celle-ci ne peut exister qu'entre les lignes raster \$30 et \$f7 — 48 et 247. Or la fenêtre de 25 rangées descend, elle, jusqu'à la ligne 250. Pour ses trois dernières lignes, nous aurions beau écrire ce que nous voulons dans \$d011, **le VIC refuserait** : aucune Bad Line n'y est possible. Privé de nouvelles couleurs, il entre dans son état de repos — et en mode graphique, l'état de repos s'affiche en **noir**.

Je le sais parce que je l'ai vu : la première version de ce programme, en 25 rangées, portait un trait noir de deux lignes en travers du bas de l'image. Pas trois : deux. Le VIC ne bascule en repos qu'une fois la rangée de pixels en cours terminée, et cette rangée-là s'achevait sur la ligne 248. La règle prédit « au plus trois », la mesure en donne deux, et l'écart lui-même s'explique — c'est le genre de moment où l'on sait qu'on a compris la machine.

La parade est alors évidente : **rétrécir la fenêtre pour rester à l'intérieur du domaine autorisé**. En 24 rangées, l'affichage court de la ligne 55 à la ligne 246, entièrement compris entre 48 et 247. Plus une seule ligne interdite, plus un seul trait noir. C'est le prix : huit lignes d'image en moins, et c'est exactement le compromis que font les images FLI que vous trouverez sur le réseau.

(Le bas de cette image m'a appris autre chose encore. Une bande noire bien plus large y traînait, sur les huit dernières lignes : mes huit matrices font 1000 octets utiles mais occupent 1024 casiers chacune, et le compteur interne du VIC dépasse 1000 dans les toutes dernières lignes d'une image. Il lisait donc les 24 octets de bourrage laissés à zéro — 24 cellules noires, exactement la largeur mesurée sur la capture. Les deux ! for supplémentaires du listing les colorient, et le bitmap a été étendu de 1000 à 1024 cellules pour la même raison. Une bande noire qui se compte en octets, ça ne s'invente pas.)

Et il faut en dire le prix : à 200 Bad Lines par image au lieu de 25, le processeur passe près de la moitié de son temps gelé. Une image FLI ne coûte pas seulement de la mémoire — elle coûte la machine. C'est pourquoi les jeux ne l'utilisent presque jamais, et les démos tout le temps.

Sous le capot — le mécanisme complet du FLI (non-incrémentation du compteur de matrice, commutation des huit matrices, l'impossibilité de commuter la Color RAM, et la cause exacte du bug des 24 pixels avec le délai de trois cycles entre les deux signaux de bus) est spécifié dans *Au cœur du métal* — Commodore 64 & Ultimate 64, chapitre 1, §16.3 ; les variantes AFLI et IFLI y sont également décrites.

Ce que vous savez faire

Arrêtons-nous, parce que le chemin mérite un regard en arrière. Vous avez commencé ce livre en ne sachant pas ce qu'était un registre. Vous venez de lire — et de comprendre — un programme qui force le matériel à relire ses couleurs deux cents fois par image, calé au cycle près sur un vol de cycles, avec un défaut d'affichage que vous savez expliquer à la microseconde.

Et vous l'avez fait avec **treize instructions**. Pas une de plus.

Au prochain chapitre

Il reste une question, et elle est de notre époque : cette machine a quarante ans, où la trouve-t-on encore ? Et surtout — car c'est là que le livre a un aveu à vous faire — sur quoi ai-je bien pu exécuter les programmes dont vous avez vu les captures ?

Chapitre 10 — La même machine en 2026

La question

Nous voici au bout du voyage, et il reste une question que vous vous êtes peut-être posée dès la première page : **où trouve-t-on, en 2026, un Commodore 64 pour vérifier tout ça ?**

Vous avez lu la même légende sous chaque image de ce livre : *capture réelle sur Ultimate 64*. C'était la promesse du chapitre 0 — toutes les images d'écran sont vraies, chacune produite par le listing imprimé juste au-dessus. Le moment est venu de dire ce qu'est cette machine, et surtout pourquoi elle ne triche pas.

Une machine décrite, pas imitée

L'objet qui a produit les captures de ce livre est un **C64 Ultimate** : une réédition du Commodore 64 vendue sous la marque Commodore, dont le cœur est une **réimplémentation FPGA**. Son firmware, « Commodore 1.0 », est un dérivé réduit du firmware Ultimate 64 de Gideon Zweijter.

Un FPGA — *field-programmable gate array* — est un circuit dont on ne fabrique pas la logique : on la **décrit**, et le circuit se reconfigure pour l'appliquer. La différence avec un émulateur logiciel est de nature. Un émulateur est un programme qui *raconte* ce qu'aurait fait une puce. Ici, il n'y a pas de programme intermédiaire : le VIC-II, le 6510 et leurs voisins sont re-décrits comme des circuits, avec le couloir unique, les deux moitiés de microseconde, le signal levé quand l'artiste réquisitionne le bus. Mêmes règles, mêmes contraintes — et, ce qui est le plus révélateur, **mêmes bizarreries**.

Car c'est ainsi qu'on démasque une imitation approximative : par les défauts. Le bug du FLI du chapitre précédent — ces vingt-quatre pixels de gauche perdus, dont la documentation de référence dit qu'« il n'y a aucun moyen de les récupérer » — est là, intact, mesuré sur cette machine. Mieux encore : il existe un décalage d'affichage si fin, si intime au fonctionnement interne du VIC, qu'il varie sur les vraies puces d'une révision de silicium à l'autre, et même avec la température. Sur ce circuit programmable, il est **déterministe** — toujours le même, mesurable une fois pour toutes. Une machine qui se contenterait d'imiter « en gros » n'aurait ni ce bug, ni ce décalage. Celle-ci les a.

Un mot d'honnêteté, parce que c'est la discipline de ce livre : les mesures dont je vais parler viennent d'un **seul exemplaire**, relevées en juin et juillet 2026. Elles ne valent pas automatiquement pour toute la série, ni pour les firmwares qui suivront. Sur votre machine : re-vérifiez.

Le turbo : soixante-quatre fois plus vite, et toujours vingt millisecondes

Cette machine sait faire une chose qu'un Commodore 64 de 1982 ne savait pas : **accélérer son processeur**. Un réglage de menu, « CPU Speed », vaut 1 MHz par défaut ; il monte jusqu'à **48 MHz** sur un Ultimate 64 et **64 MHz** sur un Elite-II. Sur le C64 Ultimate de ce livre, la mesure est faite : l'index de vitesse le plus élevé, écrit dans le casier \$d031 — un seul octet, quatre bits utiles — donne **64 MHz constants** (à condition qu'un réglage du menu autorise le programme à y toucher). Écrire dans ce casier est d'ailleurs sans effet sur un vrai C64 : c'est un registre VIC inutilisé. Le même programme peut donc demander poliment le turbo sur les deux machines.

Reprenez maintenant la métaphore du chapitre 0, celle du couloir coupé en deux moitiés. Sur un Commodore 64 d'origine, chaque microseconde offre **deux créneaux** d'accès à la réserve : un pour le VIC, un pour le processeur. Sur cette machine, la même microseconde en offre **soixante-quatre**. Et le cœur bat en permanence à cette cadence, quel que soit le réglage : l'index de vitesse ne fait qu'une chose, dire **combien de ces créneaux sont attribués au processeur**. Choisir « 1 MHz » n'est donc pas ralentir une horloge, c'est n'en réclamer qu'une petite part ; choisir le maximum, c'est les prendre tous.

Le faisceau, lui, n'a rien changé à ses habitudes. Toujours 63 cycles par ligne, toujours 312 lignes, toujours 19 656 battements, toujours **vingt millisecondes** — et c'est précisément pour cela que l'image sortie de cette machine est une vraie image PAL, que votre téléviseur accepte sans discuter. Le processeur a pris soixante-quatre fois plus de temps de travail ; le VIC, pas une microseconde de moins.

Et voici la leçon du livre, qui survit intacte à l'accélération : **même à 64 MHz, quand le VIC réquisitionne le couloir, le processeur attend**. Ce n'est pas une déduction : le gel a été mesuré sur cette machine, et sa durée — **environ 43 microsecondes** par Bad Line, le chiffre que donne aussi la documentation du turbo

— est celle du chapitre 4. Le cœur programmable fige le processeur rapide tout le temps que le VIC tient le bus. Toute technique qui *force* des Bad Lines paie donc ce prix plein, quelle que soit la vitesse affichée au menu.

Faites le compte pour le FLI du chapitre 9, et savourez : environ 200 Bad Lines forcées, à 43 microsecondes chacune, cela fait **8,6 millisecondes gelées sur une trame de 20**. Le débit utile du processeur est à peu près **divisé par deux** — au turbo comme à 1 MHz. Le titre de ce livre ne s'est jamais démenti : tout se joue toujours dans les mêmes vingt millisecondes, et il faut toujours ranger ses calculs dans les bordures.

Il y a plus beau. Ceux qui ont mesuré cette machine ont retourné la contrainte : puisque le gel se produit **au même point de chaque ligne**, il ré-aligne le programme sur le faisceau à chaque itération. Le défaut devient l'horloge. Nul besoin de code compté au cycle : la machine resynchronise d'elle-même. Vous reconnaissez le mouvement — c'est exactement celui des chapitres 7, 8 et 9. On observe une règle, on remarque qu'elle est régulière, on s'en sert.

Reste, pour être complet, qu'un interrupteur de ce même casier \$d031 permet de **désactiver le timing des Bad Lines** : le processeur continue alors de travailler pendant que le VIC tient le bus, à condition que son accès reste interne à la machine. C'est un réglage de compatibilité, pas une victoire sur le temps : le VIC garde toujours la priorité, et tout accès vers le monde extérieur — le port cartouche — reste à 1 MHz et seulement quand le VIC a rendu le couloir. Débranchez ce timing, et vous n'avez plus un Commodore 64 : vous avez une machine qui lui ressemble. La phrase du chapitre 0 tient bon jusqu'au dernier paragraphe du dernier chapitre : **dans un Commodore 64, le temps appartient à la puce vidéo**.

Le monte-charge : la REU

Un dernier accessoire, et il mérite le détour parce qu'il déplace la question du couloir.

La **REU** — *RAM Expansion Unit* — est une extension mémoire que Commodore vendait déjà à l'époque : 128 kilo-octets sur la 1700, 256 sur la 1764, 512 sur la 1750, extensibles à plusieurs mégaoctets. Sa particularité est que le processeur **ne peut pas l'adresser**. Elle n'apparaît nulle part dans les 65 536 casiers du chapitre 0.

Comment y accède-t-on, alors ? Par un **monte-charge**. La REU contient son propre contrôleur, qui déplace des blocs d'octets entre la mémoire du C64 et la sienne — dans un sens, dans l'autre, ou en échangeant les deux — sans que le processeur transporte quoi que ce soit. On lui indique une adresse ici, une adresse là-bas, une longueur, on écrit l'ordre de départ dans un casier, et le bloc voyage. Trois façons de déménager, une quatrième commande qui se contente de comparer, onze casiers de \$df00 à \$df0a : c'est tout le vocabulaire.

Le monte-charge a son tempérament, et il a été mesuré : chaque voyage coûte environ **60 microsecondes de mise en place**. Soixante microsecondes sur les vingt mille d'une trame, c'est une misère pour déménager une image entière ; c'est ruineux si l'on veut déplacer trente petites choses à chaque trame. Peu de gros transferts : oui. Beaucoup de petits : non. On croirait entendre le chapitre 4.

Sous le capot — les réglages de vitesse et leur table d'index, les registres de contrôle du turbo, le comportement des Bad Lines à haute fréquence, la table complète des registres de la REU et ses quatre commandes sont spécifiés dans *Au cœur du métal — Commodore 64 & Ultimate 64*, chapitre 6 (« L'Ultimate 64 : services »), §4, pour le turbo, et chapitre 5 (« Le REU : DMA \$df00 ») pour le monte-charge. Les mesures citées ici — les 43 microsecondes de gel à 64 MHz, les 8,6 millisecondes du FLI, les 60 microsecondes de la REU — sont dans l'**annexe A** (« Le C64 Ultimate mesuré », §A.1 et §A.5) et l'**annexe B** (§B.1), qui rapportent des relevés originaux sur l'exemplaire décrit.

La passerelle

Vous savez maintenant voir les vingt millisecondes. Vous savez qu'une couleur est une écriture, qu'un retard de dix cycles se lit à l'écran comme quatre-vingts pixels, qu'une ligne sur huit le processeur perd la parole, et qu'un cadre d'écran n'est pas une région mais un interrupteur. Ce livre s'arrête là : il montre les phénomènes.

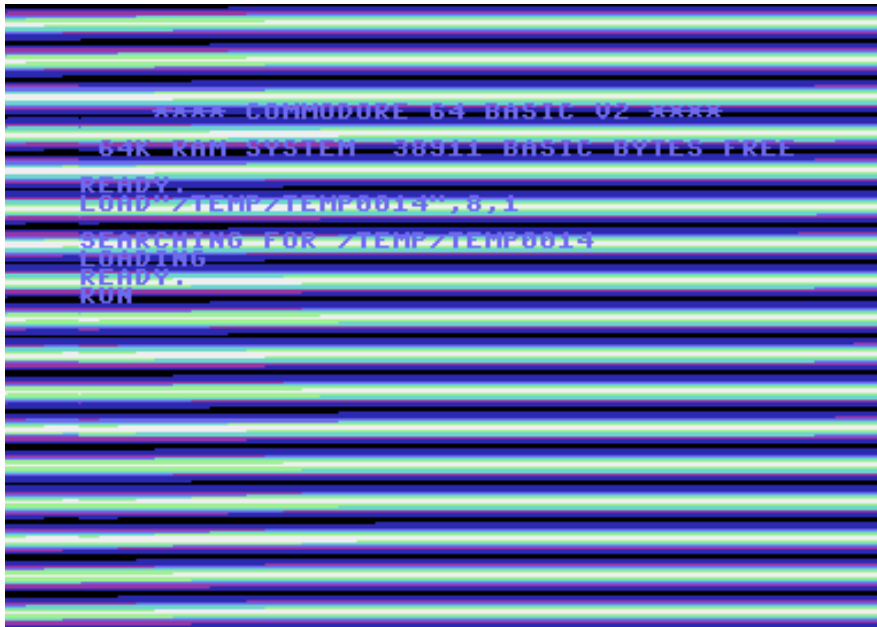
Le jour où vous voudrez les **règles exactes, cycle par cycle, avec leurs sources** — la définition normative de la Bad Line, les quarante-sept registres du VIC un par un, le diagramme d'accès mémoire de chacun des 63 cycles d'une ligne, les six règles des bascules de bordure —, ouvrez *Au cœur du métal — Commodore 64 & Ultimate 64*. Il est bâti pour ça, et pour rien d'autre : chaque affirmation y porte l'ancre du document

qui l'atteste. Il ne cherche pas à vous apprendre — il cherche à vous prouver. C'est un autre métier, et c'est pour cela qu'il a fallu deux livres.

Et si vous préférez remonter directement aux sources — l'article de Bauer, les chronogrammes de Mäkelä, les tables de cycles du 6510 —, elles sont toutes réunies à la fin de ce livre, avec l'adresse où les trouver.

La boucle est bouclée

Revenez une dernière fois au chapitre 0, à la bande-annonce que vous ne pouviez pas encore lire.



Six instructions : `sei` pour fermer la porte, `ldx $d012` pour demander où en est le faisceau, `lda couleurs,x` pour prendre dans la table ce qui était prévu pour cette ligne, deux `sta` pour peindre la bordure et le fond, `jmp` pour recommencer. Vous les lisez maintenant sans y penser, et vous savez pourquoi les vagues penchent légèrement sur les bords.

Six instructions, cinq noms différents — sur les **treize** que compte tout ce livre. Vous ne connaissez pas l'assembleur 6502 en entier, et ce n'était pas le but : vous connaissez une machine, ce qui est beaucoup plus rare. Il vous reste 19 656 battements à remplir, cinquante fois par seconde, et personne pour vous dire quoi en faire.

Annexe — quatre pages à garder sous la main

Ce livre a fait le choix de n'expliquer chaque chose qu'au moment où elle sert. C'est bon pour apprendre, moins pratique pour programmer : au bout de trois chapitres, on cherche « le numéro du rouge clair » ou « quel bit éteint l'écran ». Voici, rassemblé, tout ce que les programmes du livre utilisent.

Les seize couleurs

Elles s'écrivent dans `$d020` (bordure), `$d021` (fond), `$d027–$d02e` (sprites), ou dans la Color RAM à partir de `$d800`. Il n'y en a pas d'autres : cette palette *est* le Commodore 64.

Décimal	Hexa	Couleur	Décimal	Hexa	Couleur
0	<code>\$00</code>	noir	8	<code>\$08</code>	orange
1	<code>\$01</code>	blanc	9	<code>\$09</code>	brun
2	<code>\$02</code>	rouge	10	<code>\$0a</code>	rouge clair
3	<code>\$03</code>	cyan	11	<code>\$0b</code>	gris foncé
4	<code>\$04</code>	rose	12	<code>\$0c</code>	gris moyen
5	<code>\$05</code>	vert	13	<code>\$0d</code>	vert clair
6	<code>\$06</code>	bleu	14	<code>\$0e</code>	bleu clair
7	<code>\$07</code>	jaune	15	<code>\$0f</code>	gris clair

Les deux écritures se valent : `lda #2` et `lda #$02` donnent le même octet. Les listings de ce livre emploient la forme décimale pour les petits nombres et l'hexadécimale quand elle rend la structure plus lisible.

Au démarrage, la machine affiche du bleu (6) sur bordure bleu clair (14) — ce sont les deux valeurs que nos programmes remettent en place quand ils rendent la main.

Les huit interrupteurs de `$d011`

C'est le casier le plus important du livre : il porte les chapitres 4, 7, 8 et 9. Un seul octet, huit interrupteurs indépendants.

Bit	Nom d'usage	Ce qu'il fait
7	RST8	le neuvième bit du numéro de ligne (lecture ; voir chapitre 2)
6	ECM	mode « caractères étendus » — non utilisé dans ce livre
5	BMM	mode graphique : 0 = texte, 1 = bitmap (chapitre 9)
4	DEN	affichage allumé : 0 = écran éteint (chapitre 4)
3	RSEL	hauteur de la fenêtre : 1 = 25 rangées, 0 = 24 (chapitre 8)
2–0	YSCROLL	le cran de défilement vertical , de 0 à 7 (chapitres 4, 7, 9)

D'où les cinq valeurs qui reviennent dans les listings :

Valeur	En binaire	Ce que ça veut dire
<code>\$1b</code>	<code>%00011011</code>	l'état normal : texte, écran allumé, 25 rangées, cran 3
<code>\$0b</code>	<code>%00001011</code>	le même, écran éteint (chapitre 4)
<code>\$13</code>	<code>%00010011</code>	le normal, mais fenêtre à 24 rangées (chapitre 8)
<code>\$18</code>	<code>%00011000</code>	écran allumé, 25 rangées, cran laissé à calculer (chapitre 7)
<code>\$30</code>	<code>%00110000</code>	mode graphique , 24 rangées, cran à calculer (chapitre 9)
<code>\$38</code>	<code>%00111000</code>	le même, en 25 rangées

Remarquez `$1b` et `$0b` : un seul bit les sépare, celui de l'affichage. C'est tout le chapitre 4 en une ligne de tableau.

Les 63 cycles d'une ligne

Le livre parle de « cycle 14 », de « cycle 58 » : voici la carte. Les cycles d'une ligne sont numérotés **de 1 à 63** (machines PAL), le n° 1 commençant à l'instant où le compteur de lignes s'incrémente — à une exception

près, la ligne 0, où cet instant arrive un cycle plus tard. À huit pixels par cycle, cette numérotation est aussi une position à l'écran.

Cycles	Ce que fait le VIC
1, 3, 5, 7, 9	il va chercher les données des sprites 3 à 7
11 à 15	il rafraîchit la mémoire dynamique (une obligation électrique)
12 à 14	s'il y a Bad Line : il annonce qu'il prend le bus (le préavis de trois cycles)
15 à 54	il le prend pour de bon et lit ses 40 cellules — c'est là que le processeur est gelé (chapitre 4)
16 à 55	il lit les pixels à afficher
56, 57	il lit à vide (il lit quand même, mais jette le résultat)
58, 60, 62	il va chercher les données des sprites 0, 1 et 2 — pour la ligne suivante

Remarquez le décalage entre les deux premières lignes de ce tableau : le VIC prend le bus trois cycles avant d'en avoir besoin, parce qu'il doit laisser au processeur le temps de finir son geste. Ces trois cycles de politesse expliqueront la cicatrice du chapitre 9.

Deux conséquences que le livre utilise sans les redémontrer : l'écriture qui crée une Bad Line ne doit **pas arriver avant le cycle 14** (chapitre 9 — le cycle 14 lui-même convient), et un sprite ne coûte pas ses cycles au même endroit selon son numéro (chapitre 6).

Les notations de l'assembleur

Elles ne sont pas des instructions : le processeur ne les voit jamais. Ce sont des ordres donnés à ACME **pendant la fabrication** du programme.

Notation	Ce qu'elle fait
!to "nom.prg", cbm	le nom du fichier à produire (cbm = avec l'adresse de chargement en tête, la convention Commodore)
* = \$0810	« la suite se range à partir de cette adresse ». Ce n'est pas une multiplication : * désigne l'adresse courante
NOM = \$32	donne un nom à un nombre, pour n'avoir à le changer qu'en un seul endroit
!byte 1,2,3	pose ces octets tels quels dans le programme (des données, pas du code)
!fill 8000, \$ff	pose 8000 fois le même octet
!for i, 0, 255 { ... }	répète le bloc pour i = 0, 1, ... 255 — une boucle de fabrication , qui ne tourne pas dans la machine
!macro nom { ... } puis +nom	définit un morceau et le réclame ; l'assembleur le recopie à chaque appel
!align 255, 0	avance jusqu'à la prochaine adresse dont les 8 bits de poids faible sont nuls, c'est-à-dire au début d'une page de 256 octets (le 255 est le masque, le 0 la valeur voulue)

Et trois opérateurs, calculés eux aussi à la fabrication — ce sont les cousins des instructions and et ora du chapitre 6 :

x & 7	ne garde que les trois derniers bits de x (car 7 s'écrit %00000111)
a b	allume dans a les bits allumés dans b
x << 4	décale les bits de x de quatre rangs vers la gauche — donc range un nombre de 0 à 15 dans les quatre bits du haut de l'octet

Les treize instructions, et ce qu'elles coûtent

Tout ce livre tient dans ces treize-là. Les coûts sont ceux d'un C64 PAL, où une ligne d'écran dure 63 cycles.

Instruction	Ce qu'elle fait	Coût
lda #7	charger une valeur dans A	2
lda \$d012	charger le contenu d'un casier	4
lda table,x	charger la case n° X d'une table	4 (5 si la table franchit une page)
ldx, ldy	idem, pour les registres X et Y	mêmes coûts que lda
sta \$d020	ranger A dans un casier	4
sta table,x	ranger A dans la case n° X	5
inc \$d020	ajouter 1 au contenu d'un casier	6
dex	retirer 1 au registre X	2
cmp #\$80	comparer A à une valeur	2
and #%11110111	éteindre des bits de A	2

Instruction	Ce qu'elle fait	Coût
ora #00001000	allumer des bits de A	2
bne boucle	sauter si la comparaison n'était pas une égalité	2 si on ne saute pas, 3 si on saute
jmp boucle	sauter	3
sei	fermer la porte aux interruptions	2
rts	rendre la main à qui nous a appelés	6

Les boucles du livre, recomposées avec ce tarif : le guet d'une ligne fait 9 cycles (lda+cmp+bne), le chronomètre 9 (inc+jmp), le dégradé 19, la patience 20, le tour de FLD 21, la boucle FLI 23.

Les codes écran utilisés dans ce livre

Attention, ce ne sont **pas** les codes des caractères que vous tapez : ce sont des numéros de tiroir dans le générateur de caractères (chapitre 5).

Code	Caractère	Code	Caractère
1 à 26	les lettres A à Z, dans l'ordre	32	espace
48 à 57	les chiffres 0 à 9	81	un disque plein (le « cœur » des vieux listings)

Ajoutez 128 à n'importe lequel de ces codes pour l'obtenir en vidéo inversée.

Ce que tous les programmes supposent

Aucun listing de ce livre ne touche à \$d016, \$d017, \$d01b, \$d01c ni \$d01d. Ils comptent donc sur l'état que le KERNAL installe à l'allumage et à chaque réinitialisation : \$d011 = \$1b, \$d015 = 0 (aucun sprite allumé), \$d016 = \$08, \$d017 = 0 (pas d'étirement vertical), \$d018 = \$14, \$d01b = \$d01c = \$d01d = 0 (sprites devant le décor, une seule couleur, pas d'étirement horizontal), bordure 14, fond 6, et la banque du VIC sur les 16 premiers kilo-octets.

C'est pour cela que les créatures du chapitre 6 font 24 × 21 pixels et passent devant le texte sans qu'aucun programme ne l'ait demandé. Si une expérience se comporte autrement, réinitialisez la machine avant de la relancer.

Pour sortir d'un programme

Presque tous les programmes de ce livre qui bouclent à l'infini commencent par sei, qui coupe le clavier (l'exception est le stroboscope du chapitre 1, écrit avant que nous connaissions sei). Dans tous les cas, une fois le programme parti en boucle, la touche RUN/STOP seule ne suffit pas. Pour reprendre la main : **RUN/STOP + RESTORE** — cette combinaison passe par une autre porte, que sei ne ferme pas, et remet les registres du VIC dans leur état normal. Sur un émulateur, réinitialiser la machine fait aussi l'affaire.

Sources

Ce livre ne cite jamais un chiffre qu'il n'ait vérifié quelque part. Voici où, et ce que chaque document apporte. Presque tous sont libres d'accès et tiennent dans un fichier texte — c'est l'un des charmes de cette machine : sa documentation de référence a été écrite par des gens qui la démontraient, et elle est restée lisible.

Le volume compagnon

« **Au cœur du métal — Commodore 64 & Ultimate 64** » est la référence dont ce livre est le guide de visite : chaque encadré « Sous le capot » y renvoie par chapitre et section.

Deux précisions lui rendent justice. D'abord, **ce n'est pas une source primaire** : il a été écrit *à partir* des documents listés ci-dessous. Il les rassemble, les recoupe, les traduit, et ancre chacune de ses affirmations sur celui qui l'atteste — c'est un travail de compilation et de vérification, pas de découverte. Quand un doute porte sur un cycle, c'est vers les sources qu'il faut remonter, et il vous y conduit.

Ensuite, **il n'a aucune vocation pédagogique**, et il l'assume : il énonce la règle, il ne l'explique pas. Il se consulte, il ne se lit pas d'un bout à l'autre — un débutant s'y noierait dès la troisième page. C'est précisément pour cela que le livre que vous tenez existe. Les deux sont complémentaires et n'ont pas le même métier : ici les phénomènes, les mains dans le cambouis et la permission de se tromper ; là-bas la règle exacte, sa provenance, et rien d'autre.

Une dernière transparence, puisque nous y sommes. Ce volume-là a été fabriqué exactement comme celui-ci : **rédigé par une intelligence artificielle**, sous la direction de son éditeur humain, puis relu en croisant plusieurs modèles et confronté ligne à ligne aux documents d'origine — c'est d'ailleurs de cette manie de tout vérifier que ce livre-ci a hérité, jusqu'à mesurer ses propres captures d'écran.

Et c'est l'**ainé des deux**. « 20 millisecondes » est né de sa lecture : tout y était juste, et personne ne pouvait l'apprendre. Le livre que vous venez de finir est la réponse à ce constat — l'enfant pédagogue d'un ancêtre austère.

Le VIC-II, et le temps

- **Christian Bauer**, *The MOS 6567/6569 video controller (VIC-II) and its application in the Commodore 64*, révision du 29 septembre 2024. <https://www.cebix.net/VIC-Article.txt> Le texte de référence. C'est de lui que viennent la définition de la Bad Line (chapitre 4), les six règles des bascules de bordure (chapitre 8) et le mécanisme du FLI (chapitre 9). Attention à la révision : celle de 2024 corrige l'expansion verticale des sprites par rapport au texte de 1996 qui circule encore.
- **Marko Mäkelä**, *The memory accesses of the MOS 6569 VIC-II...* (dit « `pal.timing` »), 3 juin 1994. Fichier `pal.timing` sur <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> Les chronogrammes relevés sur matériel réel : à quel cycle exact chaque accès a lieu. C'est la carte des 63 cycles de notre annexe.
- **Pasi Ojala**, « Missing Cycles », *C=Hacking* n° 3, 1992. <http://www.zimmers.net/anonftp/pub/cbm/magazines/c=hacking/> L'article fondateur sur le vol de cycles (chapitre 6). Ses approximations ont été corrigées depuis par Mäkelä — qui le dit lui-même, et c'est une belle leçon de méthode.
- **Linus Åkesson**, « Massively Interleaved Sprite Crunch », 2016. <https://linusakesson.net/scene/lunatic/misc.php> Jusqu'où l'on peut pousser le DMA des sprites — démontré par une production qui tourne.

Le processeur

- **John West et Marko Mäkelä**, *Documentation for the NMOS 65xx/85xx Instruction Set* (dit « `64doc` »), 3 juin 1994. Fichier `64doc`, même adresse que `pal.timing`. Le coût en cycles de chaque instruction, étape par étape. Tous les comptes de ce livre — 9 cycles, 19, 20, 21, 23 — s'y vérifient.

Les puces que ce livre a laissées tranquilles

- **MOS Technology**, datasheet du **6581 SID**, 12 pages (archive 6581.zip, même adresse).
- **Wolfgang Lorenz**, *A Software Model of the CIA6526*, version 2.15, mai 1997 — le modèle qui a servi de base à la plupart des émulateurs.
- **Richard Hable**, *Programming the Commodore REU* (fichiers programming.reu et reu.registers, même adresse) — le monte-charge du chapitre 10.

La machine de 2026

- **Gideon Zweijtzer**, documentation officielle Ultimate-64 / 1541 Ultimate-II : <https://github.com/GideonZ/1541u-documentation>, rendue lisible sur <https://1541u-documentation.readthedocs.io> — l'API, les flux vidéo, le mode turbo. ⚠ Elle décrit l'**Ultimate 64** ; la machine de nos captures est un **C64 Ultimate** de marque Commodore, dont le firmware est un fork réduit. Les deux divergent sur plusieurs points : en cas de doute, c'est le matériel qui tranche, pas le document.

Le manuel d'origine

- **Commodore**, *Commodore 64 Programmer's Reference Guide*, 1982 — les codes écran (annexe B) et le chapitre « Programming Graphics », d'où viennent les tables du chapitre 5.
- **Sheldon Leemon**, *Mapping the Commodore 64* — la carte commentée de la mémoire, casier par casier.